

Diplomarbeit

EduShelf

Verfasser: Elias Welt 5AHITS
Phillip Rubak 5AHITS

Betreuer: Dipl.-Ing. Gerald Stoll

Schuljahr: 2025/26

Abgabevermerk:

Datum: 07.04.2025

übernommen von:

Höhere Technische Bundeslehranstalt Hollabrunn

Höhere Lehranstalt für Informationstechnologie

EIDESSTATTLICHE ERKLÄRUNG

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.

Für die Erstellung der Arbeit habe ich die generativen KI Tools Gemini und Claude zu folgenden Zwecken verwendet:

- Unterstützung, um neue Zugänge zu einem Thema zu finden und unterschiedliche Perspektiven zu entwickeln und damit die Ideenfindung zu erleichtern.
- Hilfe, um Textentwürfe zu erstellen, Formulierungen zu präzisieren und Fehler zu reduzieren.
- Unterstützung, um Lösungsansätze für technische Umsetzungen zu erarbeiten und die Entwicklung von Programmcode zu begleiten.
- Hilfe, um potentielle Sonderfälle und Grenzszenarien zu identifizieren und geeignete Strategien für deren Behandlung zu entwickeln.

Hollabrunn, am 07.04.2025, Elias Welt

Hollabrunn, am 07.04.2025, Phillip Rubak

1 HINWEISE

Die vorliegende Diplomarbeit wurde in Zusammenarbeit mit der Firma "X-Works" ausgeführt.

Die in dieser Diplomarbeit entwickelten Prototypen und Software-Produkte dürfen ganz oder auch in Teilen von Privatpersonen oder Firmen nur dann in Verkehr gebracht werden, wenn sie diese selbst geprüft und für den vorgesehenen Verwendungszweck für geeignet befunden haben.

Es wird keinerlei Haftung übernommen für irgendwelche Schäden, die aus der Nutzung der hier entwickelten oder beschriebenen Bestandteile des Projekts resultieren.

2 SCHLÜSSELBEGRIFFE

- EduShelf
- Backend
- Frontend
- Retrieval-Augmented Generation (RAG)
- Large Language Model (LLM)

3 DANKSAGUNGEN

Wir möchten an dieser Stelle unseren aufrichtigen Dank all jenen aussprechen, die uns während der Entstehung dieser Diplomarbeit unterstützt haben.

Unser Dank gilt XWorks, unserer Partnerfirma, die uns beim Start mit wertvollen Einblicken und praxisnaher Unterstützung begleitet haben und ebenfalls bei der Ideen und Entscheidungsfindung geholfen haben. Die Zusammenarbeit mit euch war eine ausschlaggebende Hilfe für das Projekt.

Ebenso möchten wir uns herzlich bei Herrn Stoll, unserem Diplomarbetsbetreuer, bedanken. Durch sein fundiertes Fachwissen, seine wertvollen Impulse und sein kontinuierliches Engagement hat er die Qualität unserer Ergebnisse entscheidend geprägt.

Ein herzlicher Dank gilt allen, die uns auf diesem Weg begleitet haben. Ob durch konstruktives Feedback, motivierenden Zuspruch oder die nötige Geduld – diese vielfältige Unterstützung war essenziell für das Gelingen unserer Diplomarbeit.

DIPLOMARBEIT

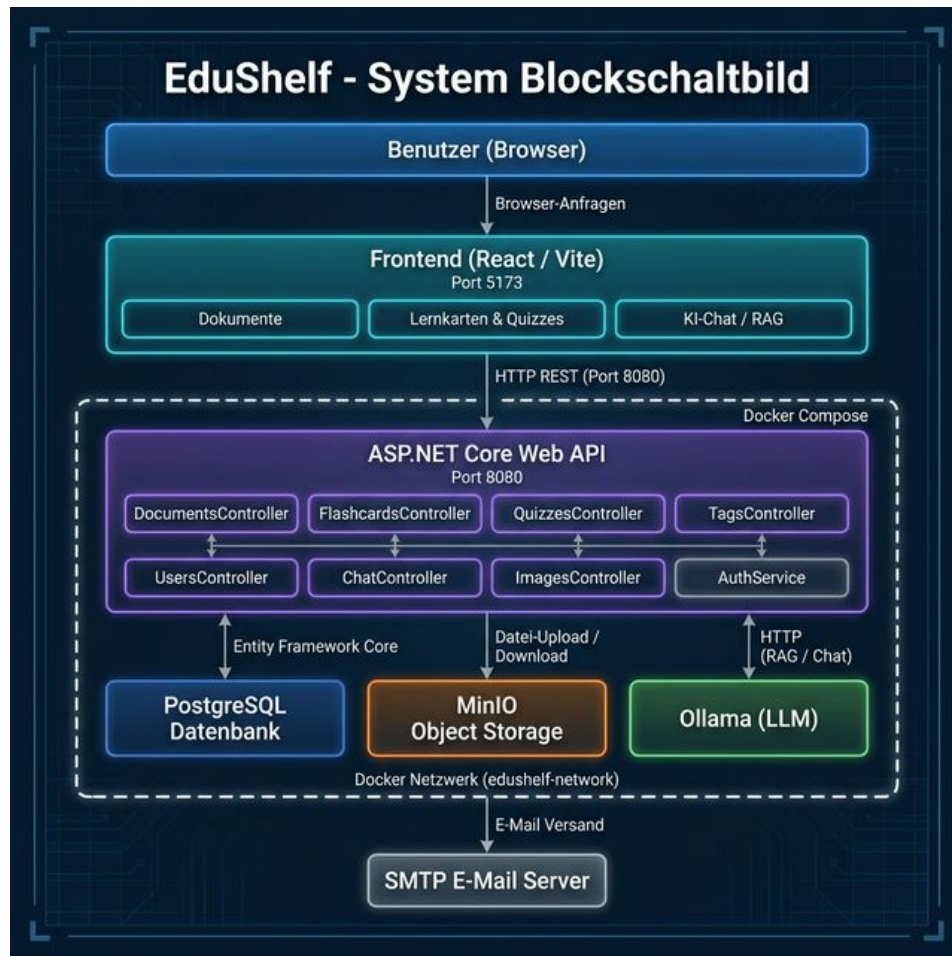
DOKUMENTATION

Name der Verfasser/innen	Elias Welt, Phillip Rubak,
Jahrgang Schuljahr	2025/26
Thema der Diplomarbeit	EduShelf
Kooperationspartner	X-Works

Aufgabenstellung	Im Rahmen dieser Diplomarbeit wird 'EduShelf' entwickelt, eine webbasierte Plattform zur Verwaltung von Bildungsunterlagen. Sie bietet Schülern und Lehrern eine zentrale Lösung zur Organisation von Dokumenten und zur KI-gestützten Informationsbeschaffung (RAG - Retrieval Augmented Generation). Das System integriert ein modernes Web-Frontend, ein Backend zur Datenverarbeitung und -speicherung sowie einen dedizierten KI-Server für intelligente Chat-Funktionalität.
------------------	--

Realisierung	Das Frontend wird mit React für eine responsive, benutzerfreundliche Oberfläche umgesetzt. Das .NET-Backend steuert Datenspeicherung und -verarbeitung, während ein Python/Ollama-basierter KI-Server die LLM-Operationen für den Chat übernimmt. Docker-Containerisierung gewährleistet einfache Bereitstellung und Skalierbarkeit.
--------------	--

Ergebnisse	Das Ergebnis ist die voll funktionsfähige Plattform 'EduShelf' mit Benutzerauthentifizierung, Dokumenten-Upload und -Verwaltung sowie einem interaktiven KI-Chat. Nutzer können ihre Dokumentenbibliothek verwalten und sofortige Antworten aus ihren Lernunterlagen erhalten – eine effektive Integration moderner Webtechnologien und KI im Bildungsbereich.
------------	--



Teilnahme an Wettbewerben, Auszeichnungen		
Möglichkeiten der Einsichtnahme in die Arbeit	HTL HollabrunnAnton Ehrenfriedstraße 10 2020 Hollabrunn	
Approbation (Datum / Unterschrift)	Prüfer/Prüferin	Direktor/Direktorin Abteilungsvorstand/ Abteilungsvorständin

DIPLOMA THESIS

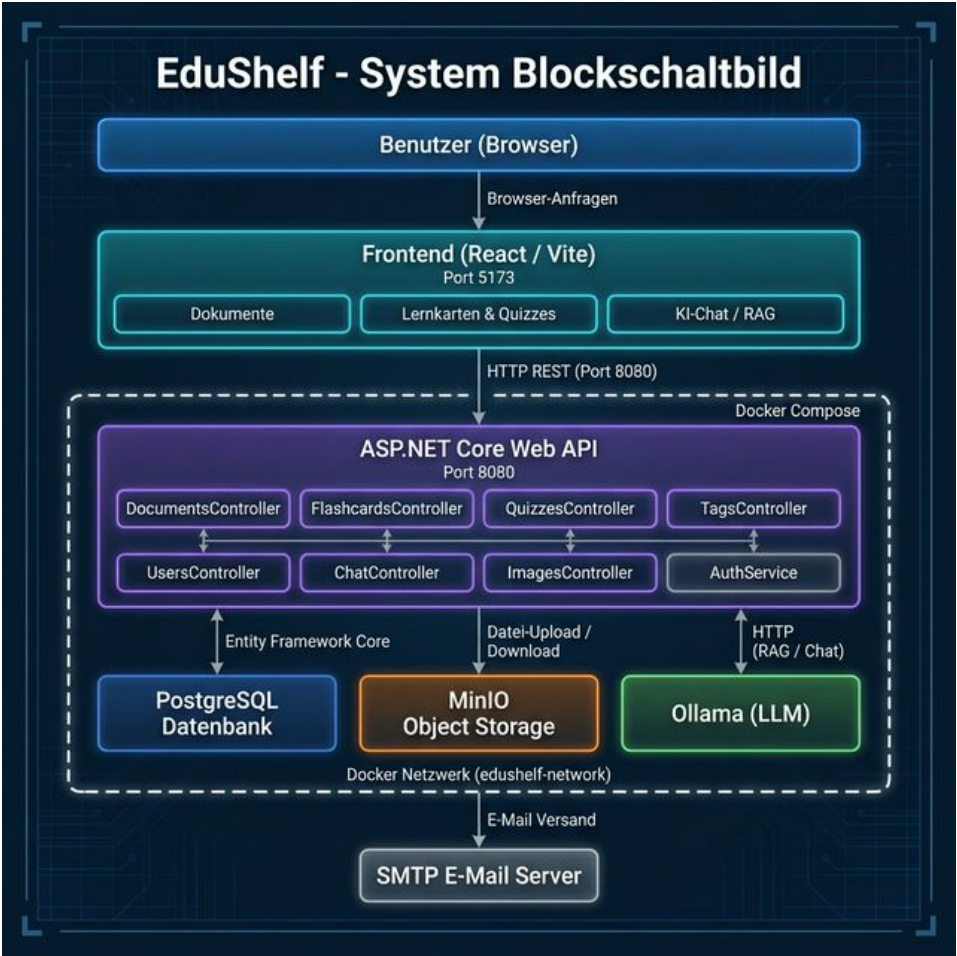
Documentation

Author(s)	Elias Welt, Phillip Rubak,
From Academic year	2025/26
Topic	EduShelf
Co-operation partners	X-Works

Assignment of tasks	This diploma project develops 'EduShelf', a web-based platform for managing educational content. It provides students and teachers with a centralized solution for organizing documents and leveraging AI-powered information retrieval (RAG - Retrieval Augmented Generation). The system integrates a modern web frontend, a backend for data processing and storage, and a dedicated AI server for intelligent chat functionality.
---------------------	---

Realisation	The frontend is built with React for a responsive, user-friendly interface. The .NET backend manages data storage and processing, while a Python/Ollama-based AI server handles LLM operations for the chat. Docker containerization ensures easy deployment and scalability.
-------------	---

Results	The result is a fully functional 'EduShelf' platform featuring user authentication, document upload and management, and an interactive AI chat. Users can manage their document libraries and obtain instant answers from their study materials, demonstrating the effective integration of modern web technologies and AI in education.
---------	--



Participation in competitions
Awards

Accessibility of
final project thesis

HTL HollabrunnAnton
Ehrenfriedstraße 10
2020 Hollabrunn

Approval (Date / Signature)

Examiner/s

Head of Department / College

Thema

EduShelf

Verlauf

- 21.05.2025: Das Thema 'xWorks KI' wurde durch Gerald Stoll angelegt.
- 09.09.2025: Sie wurden zu Thema hinzugefügt
- 09.09.2025: Sie wurden zu Thema hinzugefügt
- 15.09.2025: Thema wurde eingereicht
- 15.09.2025: Thema wurde von Gerald Stoll akzeptiert
- 15.09.2025: Thema wurde von Wilfried Trollmann akzeptiert
- 29.09.2025: Thema wurde von Wolfgang Bodei akzeptiert

Schulische Informationen

- **Schulart:** Technische, gewerbliche und kunstgewerbliche Schulen
- **Klasse:** 5AHITS
- **Abteilung:** IT
- **Schuljahr der abschließenden Prüfung:** 2025/26

Betreuung

Rolle	Name
Direktion	Wolfgang Bodei (Akzeptiert)
Abteilungsvorstand	Wilfried Trollmann (Akzeptiert)
Hauptverantwortlicher Betreuer	Gerald Stoll (Akzeptiert)

Kooperationspartner / Auftraggeber

- **Name:** X-WORKS software GmbH
- **Adresse:** Mankerstraße 24, 3380 Pöchlarn
- **URL:** <https://x-works.at/>
- **E-Mail:** info@x-works.at
- **Ansprechpartner:** Georg Ungerböck
- **Telefon:** +43 2757 24131
- **Typ:** Sonstige

Beteiligte SchülerInnen

Schüler/in	Individuelle Themenstellung	Abteilung
Elias Welt	Frontend, Frontend/Backend Schnittstelle, KI Unterstützung	IT
Phillip Rubak	Backend, Datenbank, KI, RAG	IT

Ausgangslage

Lernmaterialien entwickeln sich oft sehr schnell zu einer riesigen und unübersichtlichen Dokumentenansammlung. Dies führt dazu, dass es für die Schüler oft sehr schwierig ist daraus die wichtigsten Inhalte zu extrahieren, was in dem sonst schon sehr stressigen Schulalltag zusätzlich nochmal sehr viel Zeit kostet.

Untersuchungsanliegen

Die Digitalisierung des Bildungswesens hat zu einer starken Zunahme digitaler Lernmaterialien geführt. Diese liegen häufig verteilt auf verschiedenen Plattformen, was eine strukturierte Verwaltung und gezielte Nutzung erschwert. Gleichzeitig fehlen Lösungen, die nicht nur der Organisation dienen, sondern Lernende aktiv beim Verstehen und Verarbeiten von Inhalten unterstützen.

Die Untersuchung orientiert sich an drei Leitfragen: Wie lässt sich eine plattformunabhängige Lösung gestalten, die den Bedürfnissen von Schüler:innen, Lehrkräften und Studierenden gerecht wird? In welchem Umfang können KI-Funktionen die Arbeit mit Lernmaterialien verbessern? Welche Anforderungen an Datenschutz, Sicherheit und Barrierefreiheit sind für tatsächlichen schulischen Einsatz entscheidend?

Diese Arbeit fokussiert sich auf eine Website mit den Kernmodulen Benutzerverwaltung, Dateiverwaltung, Kategorisierung, Suche und ausgewählten KI-Funktionen. Erweiterungen wie eine mobile App oder Gamification werden als mögliche Zusatzfeatures betrachtet.

Zielsetzung

Entwicklung einer webbasierten Plattform zur Verwaltung, Kategorisierung und Nutzung von schulischen Lernmaterialien (PDF, Markdown, Textdateien). Nutzer sollen Materialien hochladen, mit Tags versehen, durchsuchen und mit einer KI interagieren können, um Inhalte besser zu verstehen oder zusammenfassen zu lassen.

Geplantes Ergebnis

Im Rahmen der Diplomarbeit entsteht eine webbasierte Plattform zur zentralen Verwaltung und Nutzung schulischer Lernmaterialien. NutzerInnen können Dateien wie PDF, Markdown oder Text hochladen, mit Tags versehen, durchsuchen und organisieren. Ein responsive Frontend sorgt für einfache Bedienung auf Desktop sowie mobilen Endgeräten.

Als Herausstellungsmerkmal gegenüber bereits vorhandenen Verwaltungsmöglichkeiten werden KI-gestützte Funktionen integriert (automatische Zusammenfassungen, Quiz- und Lernkartengenerierung sowie vereinfachte Erklärungen). Ermöglicht wird auch die direkte Interaktion mit Dokumenten via AI-Chatbot. Das geplante Ergebnis umfasst eine funktionsfähige Plattform mit Benutzerverwaltung, Datei-Upload, Suche, Tagging und ausgewählten KI-Funktionen sowie eine Dokumentation aller unserer Schritte. Ziel ist ein Tool, welches für Effizienz und Zeitersparnis steht.

Meilensteine

Meilenstein	Datum
Start Diplomarbeit	10.04.2025
Prototyp fertig	01.01.2026
Tests beenden	01.02.2026
Dokumentation	15.03.2026
Abgabe fertige Diplomarbeit	27.03.2026

Erklärung

Die unterfertigten Kandidaten/Kandidatinnen haben gemäß den geltenden schulrechtlichen Bestimmungen die Ausarbeitung einer abschließenden Arbeit (Diplomarbeit bzw. Abschlussarbeit) mit folgender Aufgabenstellung gewählt:

EduShelf - KI basierte Lernplattform

Individuelle Aufgabenstellungen im Rahmen des Gesamtprojektes:

- Elias Welt (5AHITS): **Frontend, Frontend/Backend Schnittstelle, AI, RAG (Retrieval-Augmented Generation)**
- Phillip Rubak (5AHITS): **Backend, Datenbank, AI**

Die Kandidaten/Kandidatinnen nehmen zur Kenntnis, dass die abschließende Arbeit in eigenständiger Weise und außerhalb des Unterrichtes zu bearbeiten und anzufertigen ist, wobei Ergebnisse des Unterrichtes mit einbezogen werden können, die jedenfalls als solche entsprechend kenntlich zu machen sind.

Die Abgabe der vollständigen abschließenden Arbeit hat in digitaler und in zweifach ausgedruckter Form bis spätestens **27.03.2026** beim zuständigen Betreuer/der zuständigen Betreuerin zu erfolgen.

Die Kandidaten/Kandidatinnen nehmen auch zur Kenntnis, dass ein Abbruch der abschließenden Arbeit nicht möglich ist.

Kandidaten/Kandidatinnen:

**Datum und Unterschrift bzw.
Handysignatur:**

Elias Welt (5AHITS)

Phillip Rubak (5AHITS)

Inhaltsverzeichnis

1	HINWEISE	6
2	SCHLÜSSELBEGRIFFE	7
3	DANKSAGUNGEN	8
	Abkürzungsverzeichnis	25
4	Einführung	27
4.1	Motivation	27
4.2	Umfeld und Stand der Technik	28
4.2.1	Existierende Lösungen und ihre Grenzen	28
4.2.2	Retrieval-Augmented Generation (RAG)	28
4.3	Aufgabenstellung	29
4.3.1	Funktionale Anforderungen	29
4.3.2	Nicht-funktionale Anforderungen	30
4.4	Aufbau der Arbeit	30
4.5	Entstehung der Idee	31
4.5.1	Stufe 1 – Problemidentifikation: Unkontrollierter Datei- anstieg	31
4.5.2	Stufe 2 – Lösungsansatz: Zentralisierter Speicher- und Sortierort	31
4.5.3	Stufe 3 – Erweiterung: KI-gestützte Lernunterstützung .	32
4.5.4	Stufe 4 – Synthese: EduShelf als Gesamtprodukt	32
5	Frontend Implementierung	33
5.1	Einführung und Zielsetzung	33

5.2	Technologie-Stack	34
5.2.1	React (v19)	34
5.2.2	Vite	34
5.2.3	React Router (v7)	34
5.3	Projektsetup und Konfiguration	35
5.4	Softwarearchitektur und Datenfluss	36
5.4.1	Unidirektionaler Datenfluss	36
5.4.2	Routing-Strategie	36
5.5	Datenschicht und API-Integration	37
5.5.1	API-Wrapper	37
5.6	Navigationsstruktur und Layout (MainLayout.jsx)	38
5.6.1	Outlet-Pattern und verschachtelte Routen	38
5.6.2	Rollenbasierte Navigation (RBAC)	39
5.6.3	Aktiver Zustand mit NavLink	39
5.7	Implementierung der Kernkomponenten	40
5.7.1	Komponentenübersicht	40
5.8	Authentifizierungsflow: Registrierung und E-Mail-Bestätigung	41
5.8.1	Registrierung (Register.jsx)	41
5.8.1.1	Clientseitige Validierung	42
5.8.1.2	Serverseitige Fehlerverarbeitung	42
5.8.2	Login-Prozess (Login.jsx)	43
5.8.3	E-Mail-Bestätigung	43
5.9	Das Dashboard	44
5.9.1	Architektur und Datenladen	44
5.9.2	Schnellaktionen und Statistik-Widget	45
5.10	Dateiverwaltung (Files.jsx)	46
5.10.1	Datenladen und Echtzeit-Suche	46
5.10.2	Kontextmenü und Einzelaktionen	47
5.10.3	Upload-Prozess	47
5.11	Datei-Sharing zwischen Nutzern	48
5.11.1	Share-Dialog (ShareFileModal.jsx)	48
5.11.2	Zustandsverwaltung für Shared Files	49
5.11.3	Bewertung des Ansatzes	49

5.12	Quiz-Modul (Quiz.jsx & QuizModal.jsx)	50
5.12.1	Datenstruktur	50
5.12.2	Ablauf eines Quiz	51
5.12.3	Quiz-Editor (QuizModal.jsx)	52
5.12.3.1	Dynamische Formular-Manipulation	52
5.13	Lernkarten (Flashcards.jsx)	53
5.13.1	CSS 3D-Flip-Effekt	53
5.13.2	State-Verwaltung mit einem Set	54
5.14	KI-Chat (Chat.jsx)	55
5.14.1	Implementierungsdetails	55
5.14.1.1	Request/Response-Pattern	55
5.14.1.2	Markdown-Rendering	56
5.14.1.3	Bildupload als Input	56
5.15	Admin Panel (AdminPanel.jsx)	57
5.15.1	Zweistufige Zugriffssicherung (RBAC)	57
5.15.2	Kaskadierende Datenabfragen (Master-Detail)	58
5.16	Benutzerprofil (Profile.jsx)	59
5.17	Einstellungen und Benutzerpräferenzen (Settings.jsx)	60
5.17.1	Präferenz-Initialisierung beim Laden	60
5.17.2	Theme-Umschaltung	61
5.17.3	Schließen der Einstellungsseite	61
5.18	Benutzeroberfläche und Design (UI/UX)	62
5.18.1	Styling-Strategie	62
5.18.2	Dark Mode Implementierung	62
5.19	Detaillierte Analyse ausgewählter Module	63
5.19.1	Quiz-Editor Logik (QuizModal.jsx)	63
5.19.1.1	Zustandsverwaltung	63
5.19.1.2	Dynamische Formular-Manipulation	63
5.19.1.3	Validierung und Speichern	64
5.19.2	Flashcards: 3D-Interaktion und State	64
5.19.3	Admin Panel: Kaskadierende Datenabfragen	65
5.20	Deployment und Betrieb	66
5.20.1	Docker-Containerisierung	66

5.20.2	Nginx Konfiguration	67
5.21	Herausforderungen und Lösungen	67
5.21.1	Synchronisation des UI-Status	67
5.21.2	Performance bei großen Dateilisten	67
5.22	Ausblick und Erweiterungsmöglichkeiten	68
5.23	Fazit	68
6	Backend Implementierung	70
6.1	Systemarchitektur	70
6.1.1	Presentation Layer (Controllers)	71
6.1.2	Business Logic Layer (Services)	71
6.1.3	Data Access Layer (Data)	72
6.1.4	Infrastructure Layer	72
6.2	Technologie-Stack	73
6.3	Projekt-Setup und Konfiguration	74
6.3.1	Programm-Initialisierung (Program.cs)	74
6.3.2	Konfiguration (appsettings.json)	74
6.4	Datenmodelle und Datenbankdesign	75
6.4.1	Kernelemente	75
6.4.2	Entity Relationship Diagram (ERD)	77
6.5	API Design und Controller	77
6.5.1	Ausgewählte API-Endpunkte	78
6.6	Sicherheitskonzepte	79
6.6.1	Authentifizierung & Autorisierung	79
6.6.2	Rollenbasiertes Zugriffssystem (RBAC)	80
6.6.3	E-Mail-Verifizierung	80
6.6.4	Error Handling Middleware	81
6.7	Ingestion-Pipeline (Datenverarbeitung)	81
6.7.1	Textextraktion (OCR & Parsing)	82
6.7.2	Chunking (Intelligente Segmentierung)	83
6.7.3	Embedding und Vektorspeicherung	84
6.8	RAG (Retrieval-Augmented Generation) Workflow	84
6.8.1	Systemübersicht und Datenfluss	85

6.8.2	Schritt 1: Intent-Erkennung (<code>IntentDetectionService</code>)	86
6.8.3	Schritt 2: Semantische Suche (<code>RetrievalService</code>)	87
6.8.3.1	Primäre Suche: Name-basiertes Matching	87
6.8.3.2	Fallback: Semantische Vektorsuche	87
6.8.4	Schritt 3: Prompt-Aufbau (<code>PromptGenerationService</code>) .	88
6.8.5	Schritt 4: Orchestrierung und Intent-basiertes Branching (<code>ChatService</code>)	90
6.8.6	Zusammenfassung: Retrieval-Augmented Generation [19]	90
6.9	KI-gestützte Lerninhaltsgenerierung	91
6.9.1	Generierungspipeline (5 Schritte)	91
6.9.2	Flashcard-Generierung (<code>FlashcardService</code>)	92
6.9.3	Quiz-Generierung (<code>QuizService</code>)	93
6.10	Infrastruktur & Deployment	93
6.10.1	Docker-Komposition	94
6.10.1.1	Datenbank (PostgreSQL mit pgvector)	94
6.10.1.2	KI-Inferenz (Ollama)	94
6.10.1.3	Object Storage (MinIO)	95
6.10.2	Dockerfile	96
6.11	Resilienz und Stabilität	96
6.11.1	Retry-Policies & Circuit Breaker	96
6.11.2	Timeout-Konfiguration	98
6.11.3	Graceful Degradation	98
6.11.4	Strukturiertes Logging	99
6.12	Herausforderungen und Lösungen	100
6.12.1	Blockierende Operationen bei großen Dateien	100
6.12.2	LLM Kontext-Fenster-Limits	101
6.12.3	Datenkonsistenz bei Löschoperationen	101
6.13	Qualitätssicherung und Tests	102
6.13.1	Unit Tests (xUnit + Moq)	102
6.13.1.1	Beispiel: <code>FlashcardService</code>	103
6.13.1.2	Beispiel: <code>AuthService</code>	104
6.13.2	Validatoren-Tests	105
6.13.3	Integration Tests	105

6.14 Fazit	105
6.14.1 Erreichte Ziele	106
6.14.2 Lernerkenntnisse und Herausforderungen	106
6.14.3 Mögliche Erweiterungen	107

Abkürzungsverzeichnis

ANN	Approximate Nearest Neighbor – Algorithmus zur näherungsweisen Suche des nächsten Nachbarn in hochdimensionalen Vektorräumen
API	Application Programming Interface – Programmierschnittstelle zur Kommunikation zwischen Softwarekomponenten
ASP	Active Server Pages – Webframework von Microsoft (hier: ASP.NET Core)
BLOB	Binary Large Object – Binäres Datenobjekt zur Speicherung unstrukturierter Daten
CORS	Cross-Origin Resource Sharing – Mechanismus zur kontrollierten Freigabe von Ressourcen über Domaingrenzen hinweg
CRUD	Create, Read, Update, Delete – Die vier grundlegenden Datenbankoperationen
CUDA	Compute Unified Device Architecture – Nvidia-Plattform für parallele Berechnungen auf GPUs
DI	Dependency Injection – Entwurfsmuster zur Entkopplung von Abhängigkeiten
DOM	Document Object Model – Programmierschnittstelle zur Darstellung und Manipulation von HTML-Dokumenten
DTO	Data Transfer Object – Datenobjekt zum Transport von Daten zwischen Softwareschichten
EF	Entity Framework – Object-Relational Mapper für .NET (hier: EF Core)
ERD	Entity Relationship Diagram – Diagramm zur Darstellung von Datenbank-Entitäten und deren Beziehungen
GPU	Graphics Processing Unit – Grafikprozessor, hier genutzt für KI-Berechnungen

HNSW	Hierarchical Navigable Small World – Indexstruktur für effiziente Vektorsuche
HTML	Hypertext Markup Language – Auszeichnungssprache für Webseiten
HTTP	Hypertext Transfer Protocol – Übertragungsprotokoll für das Web
JSON	JavaScript Object Notation – Leichtgewichtiges Datenaustauschformat
KI	Künstliche Intelligenz
LLM	Large Language Model – Großes Sprachmodell, z. B. Llama 3 oder Qwen3
OCR	Optical Character Recognition – Optische Zeichenerkennung zur Textextraktion aus Bildern
ORM	Object-Relational Mapper – Werkzeug zur Abbildung von Datenbanktabellen auf Objekte
RAG	Retrieval-Augmented Generation – Verfahren zur Anreicherung von LLM-Antworten durch externe Wissensquellen
RBAC	Role-Based Access Control – Rollenbasierte Zugriffskontrolle
REST	Representational State Transfer – Architekturstil für Web-APIs
S3	Simple Storage Service – Objektspeicher-Standard (Amazon, hier: MinIO-kompatibel)
SDK	Software Development Kit – Sammlung von Entwicklungswerkzeugen
SMTP	Simple Mail Transfer Protocol – Protokoll zum Versenden von E-Mails
SPA	Single Page Application – Webanwendung, die nur aus einer einzigen HTML-Seite besteht
SQL	Structured Query Language – Abfragesprache für relationale Datenbanken
UI	User Interface – Benutzeroberfläche
URL	Uniform Resource Locator – Eindeutige Adresse einer Ressource im Web
UX	User Experience – Benutzererfahrung

4 Einführung

4.1 Motivation

Der Umgang mit Lernmaterialien im Bildungsalltag ist trotz fortschreitender Digitalisierung nach wie vor nicht optimal: Dokumente liegen in verschiedenen Cloud-Diensten und Ordnern verstreut, Zusammenfassungen werden manuell erstellt und das gezielte Heraussuchen von Informationen aus umfangreichen Skripten oder Büchern kostet wertvolle Zeit. Gleichzeitig haben LLMs (Large Language Models) in den letzten Jahren ein Level erreicht, welches eine sinnvolle Integration in produktive Anwendungen ermöglicht – ohne dabei auf kommerzielle Drittanbieter-APIs angewiesen zu sein, die den Datenschutz gefährden könnten.

EduShelf adressiert genau diese Lücke: Die Plattform kombiniert zentrales Dokumentenmanagement mit KI-gestütztem Lernen in einer einzigen, datenschutzfreundlichen Webanwendung. Nutzerinnen und Nutzer können ihre Unterlagen hochladen und anschließend direkt damit interagieren – durch gezielte Fragen, automatische Lernkarten oder Multiple-Choice-Quizzes, die das LLM inklusive RAG-System aus dem Dokumenteninhalt generiert.

4.2 Umfeld und Stand der Technik

4.2.1 Existierende Lösungen und ihre Grenzen

Auf dem Markt existieren bereits Teillösungen des Problems:

- **Cloud-Speicherdienste** (z.B. Google Drive, OneDrive) ermöglichen das Ablegen und Teilen von Dokumenten, bieten aber keine inhaltliche Interaktion mit den Dateien.
- **KI-Chatbots** (z.B. ChatGPT, Google Gemini) können Fragen beantworten, haben jedoch keinen Zugriff auf eigene, private Dokumente (müssen pro Chat hochgeladen werden) – oder senden diese zur Verarbeitung an externe Server, was datenschutztechnisch problematisch ist.
- **Lernkarten-Apps** (z.B. Anki) müssen meist manuell befüllt werden und sind nicht an eine Dokumentbibliothek angebunden.

Eine Plattform, die Dokumentenverwaltung, Suche und KI-Lernhilfen in einem lokalen, datenschutzkonformen System vereint, existierte bisher nicht als Open-Source-Lösung für den Bildungsbereich.

4.2.2 Retrieval-Augmented Generation (RAG)

Der zentrale technische Ansatz von EduShelf ist *Retrieval-Augmented Generation* (RAG). Dabei wird ein LLM nicht mit seinem statischen Trainingswissen allein befragt, sondern mit relevantem Kontext aus einem externen Wissensspeicher angereichert. Konkret bedeutet das:

1. Hochgeladene Dokumente werden in kleine Textabschnitte (*Chunks*) zerlegt und als Vektoren in einer Datenbank gespeichert.
2. Bei einer Nutzeranfrage werden die semantisch passendsten Chunks per Vektorsuche abgerufen.
3. Diese Chunks werden als Kontext zusammen mit der Frage an das LLM übergeben.
4. Das LLM generiert eine passende, auf das Dokument bezogene Antwort.

Dieses Verfahren reduziert Halluzinationen und ermöglicht es, ein lokal betriebenes Modell (ohne Internetzugang) mit dokumentenspezifischem Wissen zu versorgen.

4.3 Aufgabenstellung

Im Rahmen dieser Diplomarbeit an der HTL Hollabrunn wurde die Plattform **EduShelf** vollständig konzipiert, implementiert und dokumentiert. Das System besteht aus drei Hauptkomponenten, die in jeweils 2 Arbeitsbereiche aufgeteilt wurden:

- **Web-Frontend** (Elias Welt): Eine React-basierte Single-Page-Application mit vollständiger Benutzeroberfläche für Dokumentverwaltung, Chat, Lernkarten und Quizze.
- **Backend-API** (Phillip Rubak): Eine ASP.NET Core 8 Web API, die als zentrale Schnittstelle zwischen Frontend, Datenbank, KI-Modell und Dateispeicher fungiert.

4.3.1 Funktionale Anforderungen

Das System soll folgende Kernfunktionen bereitstellen:

1. **Benutzerverwaltung:** Registrierung mit E-Mail-Verifizierung, Login, rollenbasierte Zugriffskontrolle (Student / Admin).
2. **Dokumentenverwaltung:** Upload, Ansicht, Herunterladen und Löschen von Dokumenten verschiedener Formate (PDF, DOCX, TXT). Dokumente können mit anderen Nutzern geteilt werden.
3. **KI-Chatbot (RAG):** Nutzer können Fragen zu ihren Dokumenten stellen. Das System erkennt die Nutzerintention automatisch und ruft relevante Dokumentenabschnitte ab.
4. **Lernkarten-Generierung:** Aus Dokumenteninhalten werden automatisch Flashcards (Frage-Antwort-Paare) generiert, die in einem interaktiven Review-Modus geübt werden können.
5. **Quiz-Generierung:** Das System erstellt automatisch Multiple-Choice-Quizze aus hochgeladenen Dokumenten, inklusive Auswertung und Ergebnisanzeige.
6. **Tag-System:** Dokumente und Lernkarten können mit Tags versehen und gefiltert werden.

4.3.2 Nicht-funktionale Anforderungen

- **Datenschutz:** Alle KI-Operationen laufen lokal auf dem Server (via Ollama). Es werden keine Dokumenteninhalte an externe Dienste übertragen.
- **Deployment:** Das gesamte System ist containerisiert (Docker Compose) und auf einem einzelnen Linux-Server betreibbar.
- **Erweiterbarkeit:** Die Architektur ist modular aufgebaut, sodass neue KI-Modelle oder Features ohne Umbau des Gesamtsystems integriert werden können.
- **Sicherheit:** Passwörter werden mit BCrypt gehasht, Sessions über HttpOnly-Cookies verwaltet, und alle API-Endpunkte sind standardmäßig durch Authentifizierung geschützt.

4.4 Aufbau der Arbeit

Die vorliegende Diplomarbeit ist in zwei Hauptteile gegliedert:

- **Frontend-Dokumentation** beschreibt die React-Architektur, das Komponentendesign, das Routing, die API-Anbindung und das UX-Design der Webanwendung (Elias Welt).
- **Backend-Dokumentation** beschreibt die Systemarchitektur, das Datenmodell, die REST-API, die RAG-Pipeline, die KI-Dienste, die Infrastruktur sowie die Qualitätssicherung durch automatisierte Tests (Phillip Rubak).

Beide Kapitel sind Strukturiert aufgebaut und belegen Aussagen durch Codeauszüge, Diagramme und Implementierungsdetails.

4.5 Entstehung der Idee

Die konzeptionelle Grundlage von EduShelf resultiert aus einer systematischen Analyse des schulischen Arbeitsalltags. Im Zuge der Beobachtung eigener Lernprozesse sowie der Erfahrungen aus dem Schulbetrieb kristallisierten sich wiederkehrende Ineffizienzen heraus, die als Ausgangspunkt für die Produktentwicklung dienten. Nachfolgend wird der ideelle Entstehungsprozess anhand von vier aufeinander aufbauenden Erkenntnisstufen erläutert.



Abbildung 4.1: Konzeptionelle Entwicklung der Idee – von der Problemerkennung zur Lösung

4.5.1 Stufe 1 – Problemidentifikation: Unkontrollierter Dateianstieg

Im schulischen Kontext akkumuliert sich über die Jahre hinweg eine erhebliche Menge an digitalem Lernmaterial. Mitschriften, Übungsblätter, Präsentationen und Abgaben werden typischerweise in heterogenen Strukturen auf verschiedenen Endgeräten oder Cloud-Diensten abgelegt, ohne einer konsistenten Organisationslogik zu folgen. Diese Fragmentierung führt zu einem signifikanten Mehraufwand beim Wiederauffinden relevanter Unterlagen und beeinträchtigt die Effizienz des Lernprozesses nachhaltig. Die erste konzeptionelle Erkenntnis lautet daher: Die schiere Quantität an digitalem Lernmaterial stellt ohne geeignete Verwaltungsstruktur ein wesentliches Produktivitätshemmnis dar.

4.5.2 Stufe 2 – Lösungsansatz: Zentralisierter Speicher- und Sortierort

Aus der identifizierten Problematik ergibt sich die logische Konsequenz, einen einheitlichen, zentralisierten Ablageort für sämtliche Lernmaterialien zu schaf-

fen. Dieser sollte nicht lediglich als passives Dateisystem fungieren, sondern aktive Mechanismen zur strukturierten Klassifikation und Kategorisierung bereitstellen. Konkret bedeutet dies die Möglichkeit, Dokumente mit beschreibenden Metadaten (Schlagwörter, Fächerzuordnung) zu versehen und gezielt durchsuchbar zu machen. Die Anforderung an einen solchen Speicherort geht folglich über die Funktionalität konventioneller Cloud-Lösungen hinaus, indem eine lernerorientierte Organisationsstruktur in den Mittelpunkt gestellt wird.

4.5.3 Stufe 3 – Erweiterung: KI-gestützte Lernunterstützung

Die Integration von Methoden der Künstlichen Intelligenz stellt die entscheidende konzeptionelle Erweiterung gegenüber einer reinen Dokumentenverwaltungslösung dar. Die Fähigkeit, auf Basis des individuellen Dokumentenbestands automatisiert Lernhilfen zu generieren – sei es durch kontextbezogene Fragen und Antworten im Chat-Format, durch das automatische Erstellen von Übungsaufgaben oder durch die Zusammenfassung komplexer Inhalte – hebt das System auf eine qualitativ höhere Ebene. Dieser Schritt transformiert EduShelf von einem passiven Archiv zu einem aktiven Lernbegleiter, der das gespeicherte Wissen für den Anwender unmittelbar nutzbar macht.

4.5.4 Stufe 4 – Synthese: EduShelf als Gesamtprodukt

Die vorangehenden drei Erkenntnisstufen münden in der Produktvision von EduShelf. Die Bezeichnung verbindet die Kernfunktionen symbolisch: *Edu* (lat./engl. für Bildung) und *Shelf* (engl. für Regal) verweisen auf einen strukturierten, zugänglichen Wissensraum. EduShelf vereint die zentrale Dateiverwaltung mit intelligenten Lernfunktionen zu einer kohärenten Plattform, die Schülerinnen und Schülern ermöglicht, ihren gesamten Lernprozess – vom Ablegen der Unterlagen bis zur aktiven Wissensüberprüfung – in einer einzigen Anwendung zu bündeln. Abbildung 4.1 visualisiert diesen Gedankengang als nachvollziehbaren, vierstufigen Prozess von der Problemerkennung bis zur konkreten Produktidee.

5 Frontend Implementierung

5.1 Einführung und Zielsetzung

Das Frontend der Anwendung „EduShelf“ dient als Schnittstelle zwischen dem Benutzer und den Backend-Diensten. Ziel der Implementierung war die Schaffung einer modernen, performanten und intuitiven Benutzeroberfläche, die das Lernen und Verwalten von Dokumenten so effizient wie möglich gestaltet.

Im Gegensatz zu klassischen Webanwendungen, die auf dem „Multi-Page Application“ (MPA) Prinzip basieren und bei jeder Interaktion eine neue HTML-Seite vom Server anfordern, wurde EduShelf als „Single Page Application“ (SPA) konzipiert. In einer SPA wird nur beim ersten Aufruf eine einzige HTML-Datei (`index.html`) geladen. Alle weiteren Interaktionen finden clientseitig statt, indem JavaScript (in diesem Fall React [1]) das DOM (Document Object Model) dynamisch manipuliert.

Die Vorteile dieses Architekturansatzes sind vielfältig:

- **Performance:** Da nach dem initialen Laden nur noch Rohdaten (im JSON-Format) und keine vollständigen HTML-Seiten mehr übertragen werden müssen, reduziert sich das Datenvolumen signifikant.
- **Benutzererfahrung (UX):** Übergänge zwischen verschiedenen Ansichten erfolgen nahezu latenzfrei, ohne die typische Unterbrechung eines Seiten-Reloads. Dies vermittelt das Gefühl einer nativen Desktop-Anwendung.
- **Entkopplung:** Durch die strikte Trennung von Frontend und Backend können beide Systeme unabhängig voneinander entwickelt, getestet und skaliert werden.

5.2 Technologie-Stack

Die Auswahl der Technologien erfolgte unter den Kriterien der Modernität, Verbreitung (Community-Support) und Wartbarkeit.

5.2.1 React (v19)

React ist eine von Meta entwickelte JavaScript-Library zur Entwicklung von UIs. Im Kern basiert React auf Komponenten, die wiederverwendbare Bausteine darstellen. Ein Schlüsselement von React ist das „Virtual DOM“. Anstatt bei jeder Änderung direkt das langsame DOM des Browsers zu manipulieren, erstellt React eine virtuelle Kopie im Speicher. Bei Zustandsänderungen vergleicht React den neuen virtuellen Baum mit dem alten (Diffing-Algorithmus) und berechnet die effizienteste Methode, um das reale DOM zu aktualisieren.

In EduShelf wird ausschließlich die „Functional Component“ Syntax in Verbindung mit „Hooks“ verwendet. Hooks wie `useState` oder `useEffect` ermöglichen es, Zustand und Seiteneffekte in funktionalen Komponenten zu nutzen, ohne Klassen schreiben zu müssen. Dies führt zu kompakterem und verständlicherem Code.

5.2.2 Vite

Vite (französisch für „schnell“) ist das Build-Tool der Wahl für „EduShelf“. Es löst ältere Tools wie Webpack ab, indem es während der Entwicklung native ES-Module im Browser nutzt. Das bedeutet, dass der Browser die Importe selbst auflöst, anstatt dass der gesamte Code vor jedem Start gebündelt werden muss. Dies führt zu extrem schnellen Server-Startzeiten, selbst bei wachsender Projektgröße. Für die Bereitstellung der Anwendung nutzt Vite im Hintergrund Rollup, um hochoptimierte, minifizierte Bundles zu erstellen.

5.2.3 React Router (v7)

Da eine SPA (= Single Page Application) physisch nur aus einer Seite besteht, muss die Navigation simuliert werden. **React Router** überwacht die URL und rendert basierend auf definierten Regeln die entsprechenden Komponenten. Dies ermöglicht Deep-Linking (Lesezeichen auf Unterseiten) und die Verwendung der Browser-Historie (Zurück-Button), obwohl die Seite nie neu geladen wird.

5.3 Projektsetup und Konfiguration

Das Projekt wurde mit dem Befehl `npm create vite@latest` initialisiert [2]. Die Konfiguration befindet sich in der Datei `vite.config.js`.

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vite.dev/config/
5 export default defineConfig({
6   plugins: [react()],
7   server: {
8     host: true,
9     port: 5173,
10    proxy: {
11      '/api': {
12        target: process.env.VITE_API_TARGET || 'http://localhost:5015',
13        changeOrigin: true
14      }
15    }
16  }
17 })
```

Listing 5.1: Auszug aus der Vite-Konfiguration

Ein kritischer Aspekt dieser Konfiguration ist der **Proxy**: Um während der Entwicklung Cross-Origin Resource Sharing (CORS) [3] Probleme zu vermeiden, leitet der Vite-Entwicklungsserver alle Anfragen, die mit `/api` beginnen, an das Backend (Port 5015) weiter. Die Option `changeOrigin: true` manipuliert dabei den Host-Header der Anfrage so, als käme sie direkt vom Backend-Server selbst.

Die Projektstruktur im Ordner `src` folgt einer funktionalen Aufteilung:

- `components/`: Beinhaltet UI-Komponenten (z.B. `Chat.jsx`, `Quiz.jsx`). Jede Komponente hat idealerweise ihre eigene CSS-Datei (z.B. `Chat.css`) für Styling
- `services/`: Inkludiert die gesamte API-Kommunikation
- `assets/`: Für statische Dateien

5.4 Softwarearchitektur und Datenfluss

5.4.1 Unidirektionaler Datenfluss

React erzwingt einen unidirektionalen Datenfluss („One-Way Binding“). Daten fließen immer von Parent- zu Child-Komponenten mittels sogenannter „Props“. Um Daten von einem Child zurück zum Parent zu senden, übergibt die Parent-Komponente eine Callback-Function an das Child.

5.4.2 Routing-Strategie

Das Routing in `App.jsx` definiert die Struktur der Anwendung [4]. Es wird zwischen offenen Routen (Login, Register) und geschützten Routen unterschieden.

```

1 <Routes>
2   { /* Öffentliche Routen */ }
3   <Route path="/login" element={<Login setLoggedInUser={setLoggedInUser} />}
4     />
5
6   { /* Geschuetzte Routen - erfordern Login */ }
7   <Route path="/*" element={
8     loggedInUser ? (
9       <Routes>
10        <Route element={<MainLayout handleLogout={handleLogout} user={
11          loggedInUser} />} />
12        <Route index element={<MainDashboard />} />
13        <Route path="files" element={<Files />} />
14        <Route path="chat" element={<Chat />} />
15        { /* ... weitere Routen ... */ }
16      </Routes>
17    ) : (
18      <Navigate to="/login" />
19    )
20  } />
21 </Routes>

```

Listing 5.2: Routing-Struktur in `App.jsx`

Die Verwendung von verschachtelten Routen (`<Route element=<MainLayout ...>>`) erlaubt es, das Grundgerüst (Header, Navigation) einmalig zu definieren. Die Child-Komponenten werden dann an der Stelle des `<Outlet />` Platzhalters in `MainLayout.jsx` gerendert.

5.5 Datenschicht und API-Integration

Die gesamte Kommunikation mit dem Backend wird über das Modul `src/services/api.js` abgewickelt. Diese Methode, oft als "SService Layer Pattern" bezeichnet, hat den Vorteil, dass Komponenten keine Details über URLs oder HTTP-Header kennen müssen.

5.5.1 API-Wrapper

Das `api`-Objekt fungiert als Wrapper um die native `fetch`-API. Es implementiert globale Verhaltensweisen:

1. **Basis-URL:** Alle Anfragen sind relativ zu einer zentral konfigurierten Basis-URL
2. **Credentials:** Durch `credentials: 'include'` sendet der Browser automatisch Cookies (für die HttpOnly Session-Cookies) mit jeder Anfrage mit.
3. **Content-Type Handling:** JSON-Payloads werden automatisch serialisiert und der korrekte Header gesetzt.
4. **Zentrale Fehlerbehandlung:** Antworten mit einem HTTP-Statuscode außerhalb des 200er-Bereichs lösen automatisch eine Exception aus. Dies vereinfacht die Fehlerbehandlung in den Komponenten, da `try/catch`-Blöcke verwendet werden können.
5. **Spezifische Methoden:** Methoden wie `createChatSession` oder `uploadDocument` bilden die Logik ab. Letzteres nutzt `FormData`, um Datei-Uploads (multipart/form-data) zu ermöglichen.

```

1 get: async (endpoint, options = {}) => {
2   const response = await fetch(`${API_BASE_URL}${endpoint}`, {
3     credentials: 'include',
4     ...options,
5   });
6   if (!response.ok) {
7     // Fehlerbehandlung: Logging und Werfen eines Errors
8     throw new Error('API Error: ${response.status}');
9   }
10  return await response.json();
11 },

```

Listing 5.3: GET-Methode im API-Service

5.6 Navigationsstruktur und Layout (MainLayout.jsx)

Die Navigation erfolgt durch die Komponente `MainLayout.jsx`. Sie stellt sicher, dass die Navigationsleiste stets sichtbar bleibt, während sich der Inhaltsbereich dynamisch verändert.

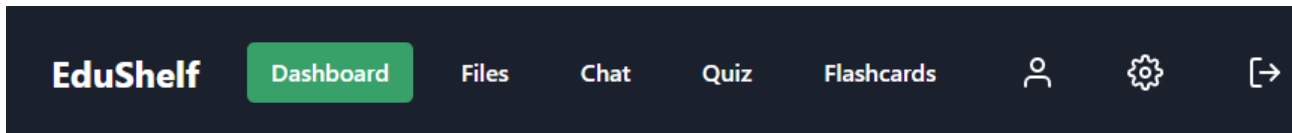


Abbildung 5.1: Navigationsleiste von EduShelf (Dark-Mode)

5.6.1 Outlet-Pattern und verschachtelte Routen

Das Layout nutzt das `<Outlet />`-Konzept von React Router [4]. Dieses Schema erlaubt es, die Navigationsleiste und den Inhaltsbereich strikt voneinander zu trennen: Das Layout wird einmal gerendert und bleibt dann, während React Router die jeweils aktive Child-Komponente an der `<Outlet />`-Stelle einfügt. Dadurch wird unnötiges Re-Rendern vermieden und die Performance optimiert.

```

1 const MainLayout = ({ handleLogout, user }) => {
2   return (
3     <div className="app-container">
4       <header className="navbar">
5         { /* Navigationsleiste -- wird nie neu gerendert */ }
6         <nav className="navbar-nav">
7           <NavLink to="/">Dashboard</NavLink>
8           <NavLink to="/files">Files</NavLink>
9           { /* ... */ }
10        </nav>
11      </header>
12      <main className="main-content">
13        <Outlet /> { /* Hier wird die aktive Seite eingefuegt */ }
14      </main>
15    </div>
16  );
17 };

```

Listing 5.4: Outlet-Pattern in MainLayout.jsx

5.6.2 Rollenbasierte Navigation (RBAC)

Die Sichtbarkeit des Menüpunkts *Admin Panel* wird direkt in der `MainLayout.jsx`-Komponente gesteuert. Eine Abfrage überprüft, ob das übergebene `user`-Objekt die Rolle `'Admin'` im enthält:

```
1 {user && user.roles && user.roles.includes('Admin')} && (
2   <NavLink to="/admin">Admin Panel</NavLink>
3 )}
```

Listing 5.5: Rollenbasierte Anzeige des Admin-Menüpunkts

Diese Überprüfung findet ausschließlich clientseitig statt und dient dazu zu vermeiden, dass Benutzer unerwünschterweise Rechte erlangen, die sie eigentlich nicht haben sollten. Die eigentliche Zugriffskontrolle auf admin-spezifische API-Endpunkte erfolgt allerdings serverseitig, sodass ein manuelles Manipulieren der URL durch den Nutzer keine sicherheitsrelevanten Auswirkungen hat.

5.6.3 Aktiver Zustand mit NavLink

React Router stellt die Komponente `NavLink` bereit, welche gegenüber dem einfachen `Link` den Vorteil bietet, den aktuell aktiven Zustand zu kennen. Über eine Callback-Funktion kann die CSS-Klasse des Elements dynamisch gesetzt werden:

```
1 <NavLink
2   to="/files"
3   className={({ isActive }) => (isActive ? 'active' : '')}>
4 >
5   Files
6 </NavLink>
```

Listing 5.6: Dynamische CSS-Klasse bei aktivem Navigationselement

Der Parameter `isActive` wird von React Router automatisch auf `true` gesetzt, wenn die aktuelle URL mit dem `to`-Attribut des Links übereinstimmt. Die CSS-Klasse `active` hebt den aktuell ausgewählten Menüpunkt visuell hervor und verbessert so die UX, da man immer genau erkennen kann wo man sich aktuell befindet.

5.7 Implementierung der Kernkomponenten

Dieses Kapitel gibt einen Überblick über die zentralen Komponenten von EduShelf. Jede Komponente ist als eigenständige `.jsx`-Datei realisiert und über React Router als eigene Route erreichbar. Die detaillierte Betrachtung der Tabs – Dateiverwaltung, Quiz, Lernkarten, Chat, Admin und Profil – erfolgt in den nachfolgenden Abschnitten.

5.7.1 Komponentenübersicht

Die folgende Tabelle gibt einen Überblick über alle Hauptkomponenten und ihre Zuständigkeit:

Komponente	Zuständigkeit
Login.jsx	Anmeldung, Session-Initialisierung
Register.jsx	Account-Erstellung, clientseitige Validierung
MainDashboard.jsx	Startseite, Statistiken, Schnellaktionen
Files.jsx	Dateiübersicht, Upload, Suche, Teilen
Quiz.jsx	Quiz-Übersicht und Durchführung
QuizModal.jsx	Quiz-Editor (Erstellen / Bearbeiten)
Flashcards.jsx	Lernkarten mit 3D-Flip-Effekt
Chat.jsx	KI-Chat mit Markdown Support und Bildupload
AdminPanel.jsx	Benutzerverwaltung (nur Admin)
Profile.jsx	Profilübersicht des eingeloggten Nutzers
Settings.jsx	Dark/Light-Mode und UI-Präferenzen

Tabelle 5.1: Übersicht der Kernkomponenten von EduShelf

5.8 Authentifizierungsflow: Registrierung und E-Mail-Bestätigung

Der Authentifizierungsprozess von EduShelf umfasst drei aufeinander aufbauende Schritte: die Erstellung eines neuen Benutzerkontos, die Verifikation der E-Mail-Adresse sowie den eigentlichen Login. Dieser Ablauf stellt sicher, dass nur Nutzer mit einer verifizierten Identität Zugang zur Plattform erhalten.

5.8.1 Registrierung (Register.jsx)

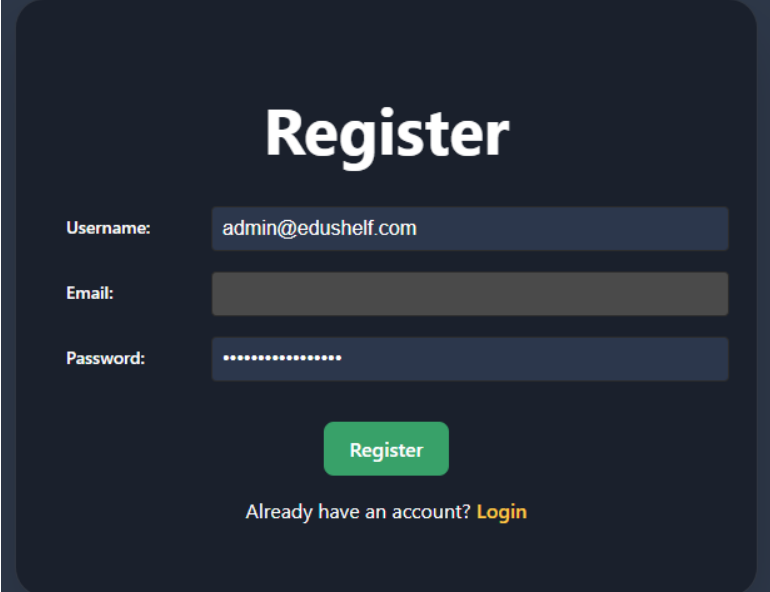
The image shows a dark-themed registration form titled "Register" in white text. It features three input fields: "Username:" with the value "admin@edushelf.com", "Email:" (empty), and "Password:" (masked with dots). Below the fields is a green "Register" button. At the bottom, there is a link that says "Already have an account? Login" in yellow text.

Abbildung 5.2: Registrierung von EduShelf im Dark-Mode

Die Komponente `Register.jsx` realisiert ein zweistufiges Validierungskonzept. Zunächst wird die Benutzereingabe vollständig clientseitig geprüft, bevor anschließend eine Anfrage an den Server gesendet wird. Dies reduziert unnötige Netzwerkanfragen und gibt dem Nutzer sofortiges Feedback.

5.8.1.1 Clientseitige Validierung

Bei diesem Teil der Validierung werden alle Felder nach klar definierten Regeln überprüft:

```
1 const validate = () => {
2   const newErrors = {};
3   if (!username || username.length < 3 || username.length > 20) {
4     newErrors.username = 'Username must be between 3 and 20 characters.';
5   }
6   if (!email || !/\S+@\S+\.\S+/.test(email)) {
7     newErrors.email = 'A valid email address is required.';
8   }
9   if (!password || password.length < 6) {
10    newErrors.password = 'Password must be at least 6 characters long.';
11  }
12  setErrors(newErrors);
13  return newErrors;
14 };
```

Listing 5.7: Clientseitige Validierung der Eingaben

Diese Methode setzt auf Regex zur E-Mail-Validierung und Längenprüfungen für Benutzernamen und Passwort. Nur wenn `validate()` ein leeres Fehlerobjekt zurückgibt, wird die Backend-Anfrage ausgelöst.

5.8.1.2 Serverseitige Fehlerverarbeitung

Serverseitige Fehler (z. B. ein bereits verwendeter Benutzername) werden strukturiert verarbeitet. Das Backend sendet ein Fehlerobjekt mit feldspezifischen Nachrichten zurück, welches das Frontend auflöst und den entsprechenden Feldern zuordnet:

```
1 if (error.response?.data?.errors) {
2   const backendErrors = {};
3   for (const key in error.response.data.errors) {
4     // Schlüssel mit Kleinbuchstaben normalisieren (z.B. "Username" -> "
       username")
5     const formattedKey = key.charAt(0).toLowerCase() + key.slice(1);
6     backendErrors[formattedKey] = error.response.data.errors[key].join(' ');
7   }
8   setErrors(backendErrors);
9 }
```

Listing 5.8: Verarbeitung von Backend-Validierungsfehlern

5.8.2 Login-Prozess (Login.jsx)

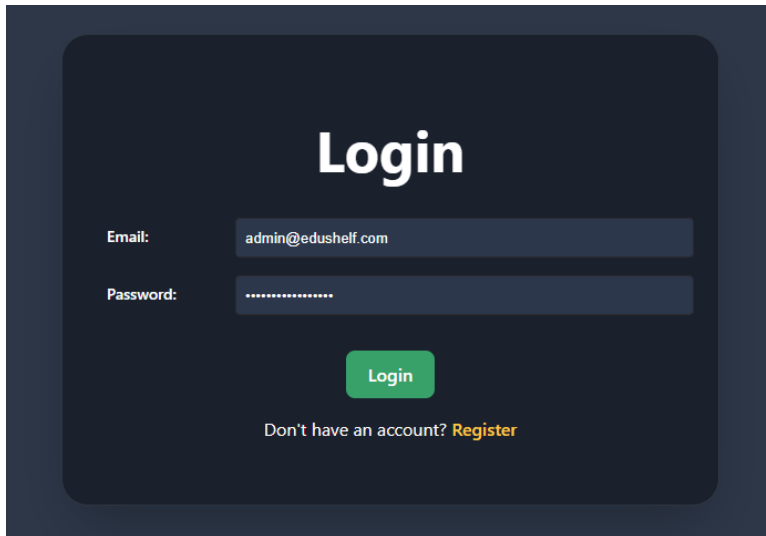


Abbildung 5.3: Login-Ansicht von EduShelf im Dark-Mode

Der Login-Prozess ist bewusst in zwei Abfragen gegliedert: Zuerst wird ein POST-Request an `/Users/login` gesendet. Bei Erfolg antwortet das Backend mit einem `HttpOnly`-Session-Cookie [5] und den Benutzerdaten. Unmittelbar danach erfolgt ein zweiter Aufruf an `/Users/{id}/roles`, da die Rollenzuweisung (*Admin*, *User*) bestimmt, welche Menüpunkte in der Navigation sichtbar sind.

Das kombinierte Benutzerobjekt (Daten + Rolle) wird im `localStorage` [6] hinterlegt, um den Anmeldestatus über Browser-Refreshes hinweg zu erhalten. Diese Persistenzmaßnahme dient ausschließlich der UI-Zustandsverwaltung – der tatsächliche Sicherheitsanker ist der `HttpOnly`-Cookie. Da der `localStorage` vom Nutzer manipulierbar ist, wird er explizit nicht als Sicherheitsmechanismus genutzt: Bei jeder Initialisierung der App erfolgt ein Abgleich mit `/Users/me` gegen den serverseitigen Cookie.

5.8.3 E-Mail-Bestätigung

Nach der Registrierung erhält der Nutzer eine Bestätigungs-E-Mail. Der enthaltene Link führt zur Komponente `VerifyEmail.jsx`, welche den Token aus der URL extrahiert und an den Backend-Endpunkt zur Verifikation übermittelt. Da das Backend den Link selbst generiert, enthält das Frontend lediglich die Anzeigelogik für die Erfolgs- bzw. Fehlermeldung – eine klare Umsetzung des Single-Responsibility-Prinzips.

5.9 Das Dashboard

Die Startseite der Anwendung, realisiert in der Komponente `MainDashboard.jsx`, stellt nach erfolgreicher Authentifizierung die primäre Ansicht für den Benutzer dar. Sie verfolgt das Ziel auf einen Blick die relevantesten Informationen und die häufigsten Aktionen darzustellen. Das Dashboard bündelt Schnellaktionen, kürzlich hinzugefügte Dateien sowie aussagekräftige Statistiken.

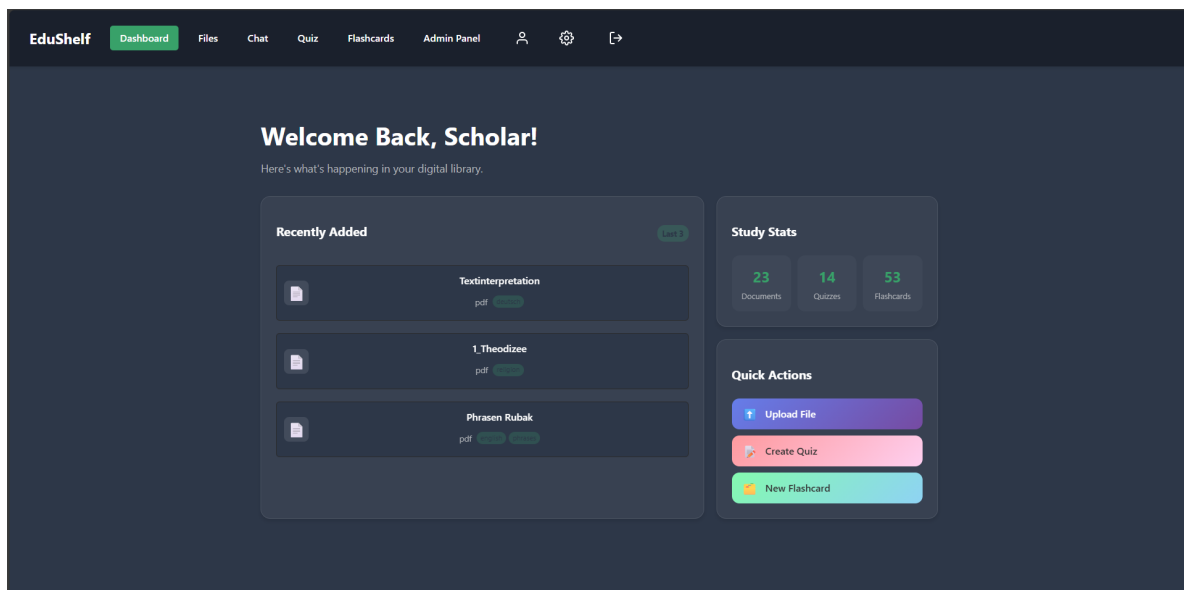


Abbildung 5.4: Dashboard von EduShelf im Dark-Mode

5.9.1 Architektur und Datenladen

Ein zentraler Vorteil der beim Dashboard verwendeten Architektur ist das parallele Laden aller benötigten Daten. Anstatt Dokumente, Quizzes und Lernkarten sequenziell abzurufen – was die Ladezeit mit jeder weiteren Anfrage erhöhen würde – wird `Promise.all()` [7] eingesetzt. Diese Methode startet alle asynchronen Anfragen gleichzeitig und wartet, bis die Letzte von ihnen abgeschlossen ist.

```

1 const fetchData = async () => {
2   try {
3     const [docsData, quizzes, flashcards] = await Promise.all([
4       getDocuments(),
5       getQuizzes(),
6       getFlashcards()
7     ]);
8     // Alle Daten sind gleichzeitig verfuegbar
9     setStats({
10      documents: docsData.totalCount || docs.length,
11      quizzes:   quizzes.totalCount   || quizzes.length,
12      flashcards: flashcards.totalCount || flashcards.length
13    });
14   } catch (error) {
15     console.error('Error fetching dashboard data:', error);
16   }
17 };

```

Listing 5.9: Paralleles Datenladen mit Promise.all im Dashboard

Das Ergebnis ist eine messbar kürzere Wartezeit im Vergleich zu sequenziellen Aufrufen, da die Gesamtdauer der Zeit der langsamsten Einzelanfrage entspricht, nicht der Summe aller Anfragen.

5.9.2 Schnellaktionen und Statistik-Widget

Gegliedert ist das Dashboard in drei Bereiche:

1. **Recently Added:** Zeigt die drei zuletzt hochgeladenen Dokumente mit Name, Typ und zugewiesenen Tags an. Ein Klick öffnet die Datei-Vorschau über `FileViewer`.
2. **Study Stats:** Zeigt die Anzahl von Dokumenten, Quizzes und Lernkarten. Diese Statistiken werden bei jeder Aktion (Upload, Erstellen, Löschen) durch erneuten Aufruf von `fetchData()` aktualisiert.
3. **Quick Actions:** Die drei Schnellaktionen-Schaltflächen ermöglichen das unmittelbare Erstellen eines Uploads, eines Quiz oder einer Lernkarte, ohne dass dafür die entsprechende Unterseite aufgerufen werden muss. Die zugehörigen Modals werden direkt gerendert und angezeigt.

Dieses Konzept minimiert die Anzahl der notwendigen Navigationsschritte für häufig ausgeführte Aktionen und entspricht der Zielsetzung einer effizienten Lernerfläche.

5.10 Dateiverwaltung (Files.jsx)

Die Dateiverwaltung ist ein zentrales Element von EduShelf und wird durch `Files.jsx` realisiert. Sie bietet eine vollständige Übersicht aller hochgeladenen Dokumente und bündelt alle dateibasierten Aktionen – Upload, Suche, Filter, Vorschau, Teilen, Download und Löschen – in einer einzigen Ansicht.

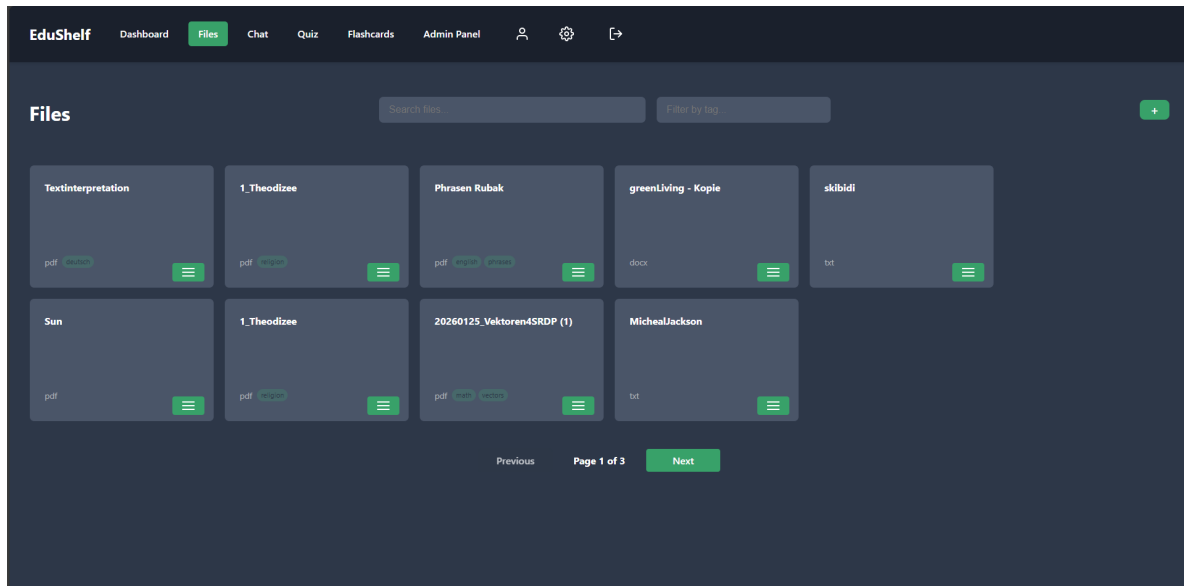


Abbildung 5.5: Dateiübersicht von EduShelf im Dark-Mode

5.10.1 Datenladen und Echtzeit-Suche

Beim Laden der Komponente ruft ein `useEffect`-Hook die vollständige Dateiliste vom Server ab und speichert sie lokal im State. Die Suchfunktion operiert ausschließlich auf dieser lokalen Kopie – es wird kein neuer API-Aufruf bei jeder Eingabe ausgelöst.

```
1 const filteredFiles = files.filter(file =>
2   file.name.toLowerCase().includes(searchTerm.toLowerCase())
3 );
```

Listing 5.10: Clientseitige Echtzeit-Suche in der Dateiliste

Auch das Filtern erfolgt in Echtzeit über `Array.filter()`. Dieses Vorgehen ist sowohl performant als auch benutzerfreundlich: Da alle Daten bereits geladen sind, erfolgt das Filtern ohne wahrnehmbare Verzögerung, unabhängig von der Netzwerkqualität.

5.10.2 Kontextmenü und Einzelaktionen

Jede Datei in der Liste besitzt ein Kontextmenü, das über ein Drei-Punkte-Icon geöffnet wird. Der State `activeMenuId` speichert die ID der Datei, deren Menü aktuell geöffnet ist. Dadurch wird sichergestellt, dass immer nur ein Kontextmenü gleichzeitig sichtbar ist:

```
1 const toggleMenu = (fileId) => {  
2   setActiveMenuId(prev => (prev === fileId ? null : fileId));  
3 };
```

Listing 5.11: Menüsteuerung über `activeMenuId`

Das Menü bietet folgende Optionen:

- **Preview:** Öffnet die Vorschau mittels `FileViewer` direkt im Browser
- **Edit:** Ermöglicht das Bearbeiten von Tags
- **Share:** Öffnet das `ShareFileModal` zur Weitergabe an andere Nutzer
- **Delete:** Löscht die Datei und aktualisiert die Liste
- **Download:** Ermöglicht den Download von Files

5.10.3 Upload-Prozess

Der Dateiupload wird über ein Modal (`UploadDialog.jsx`) realisiert. Der Nutzer wählt eine Datei aus, die zusammen mit der User-ID als `multipart/form-data` [8] an den Server gesendet wird. Nach erfolgreichem Upload wird `fetchFiles()` erneut aufgerufen, um die Ansicht ohne manuellen Seitenreload zu aktualisieren. Außerdem gibt es die Möglichkeit Tags hinzuzufügen, nach denen die Dateien später gefiltert werden können

5.11 Datei-Sharing zwischen Nutzern

Eine weitere wesentliche Funktion von EduShelf ist die Möglichkeit, Dokumente mit anderen Nutzern zu teilen. Der komplette Sharing-Prozess wird durch die Komponente `ShareFileModal.jsx` in Verbindung mit der Dateiansicht `Files.jsx` realisiert.

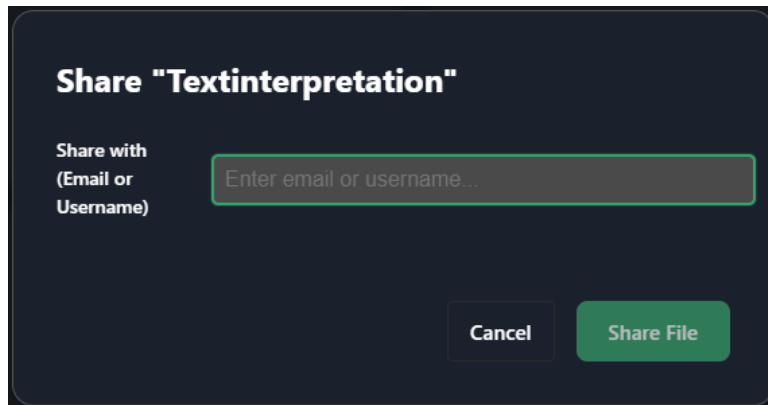


Abbildung 5.6: Share-File-Dialog von EduShelf im Dark-Mode

5.11.1 Share-Dialog (ShareFileModal.jsx)

Der Share-Dialog ist als Pop-Up-Modal implementiert. Der Nutzer gibt die E-Mail-Adresse oder den Benutzernamen der Zielperson ein. Da beide Eingabeformate akzeptiert werden, entfällt die Notwendigkeit, die genaue Identifikation des Empfängers zu kennen – das Backend löst die Zuordnung auf.

```

1  const ShareFileModal = ({ isOpen, onClose, onShare, fileName }) => {
2    const [emailOrUsername, setEmailOrUsername] = useState('');
3    const [isLoading, setIsLoading] = useState(false);
4
5    const handleSubmit = async (e) => {
6      e.preventDefault();
7      if (!emailOrUsername.trim()) return;
8      setIsLoading(true);
9      try {
10       await onShare(emailOrUsername);
11       setEmailOrUsername('');
12     } finally {
13       setIsLoading(false);
14     }
15   };
16 };

```

Listing 5.12: Formularlogik im ShareFileModal

Während des Sendevorgangs wird der Submit-Button deaktiviert (`disabled=isLoading`) und der Text wechselt von *Share File* zu *Sharing...*. Dieses Muster verhindert mehrere Einreichungen und gibt dem Nutzer visuelles Feedback über den schon laufenden Prozess.

5.11.2 Zustandsverwaltung für Shared Files

Empfangene Freigaben werden im Dateibereich des Empfängers mit einem visuellen Attribut (`isShared: true`) angezeigt, um klarzumachen, dass es kein eigenes File ist. Der Nutzer hat die Möglichkeit, eine Freigabe zu akzeptieren oder abzulehnen. Diese Entscheidung wird anschließend serverseitig gespeichert, sodass der Status geräteübergreifend abrufbar bleibt:

```
1 // Akzeptieren einer Freigabe
2 const handleAcceptShare = async (docId) => {
3   await api.patch(`/files/shared/${docId}/accept`);
4   fetchFiles(); // Listenaktualisierung auslösen
5 };
6
7 // Ablehnen einer Freigabe
8 const handleRejectShare = async (docId) => {
9   await api.patch(`/files/shared/${docId}/reject`);
10  fetchFiles();
11 };
```

Listing 5.13: Serverseitige Speicherung der Shares via API

5.11.3 Bewertung des Ansatzes

Die Entscheidung, den Accept/Reject-Status serverseitig zu speichern, stellt sicher, dass der Share-Status geräteübergreifend verfügbar ist. Akzeptiert ein Nutzer eine Freigabe auf einem Gerät, ist diese Entscheidung auch auf allen anderen Geräten, die mit diesem Account eingeloggt sind, sofort sichtbar. Dadurch entfallen die früheren Einschränkungen einer rein clientseitigen Lösung und die User Experience erhöht sich deutlich. Die API-Aufrufe erfolgen asynchron, sodass die Benutzeroberfläche mit den aktuellen Daten vom Server aktualisiert wird.

5.12 Quiz-Modul (Quiz.jsx & QuizModal.jsx)

Das Quiz-Modul bündelt alle Funktionen für interaktives Lernen durch Abfragen. Es besteht aus zwei eng verzahnten Komponenten: `Quiz.jsx` übernimmt die Übersicht aller vorhandenen Quizzes sowie die vollständige Durchführungslogik, während `QuizModal.jsx` als Modal für Create- und Update-Operationen dient.

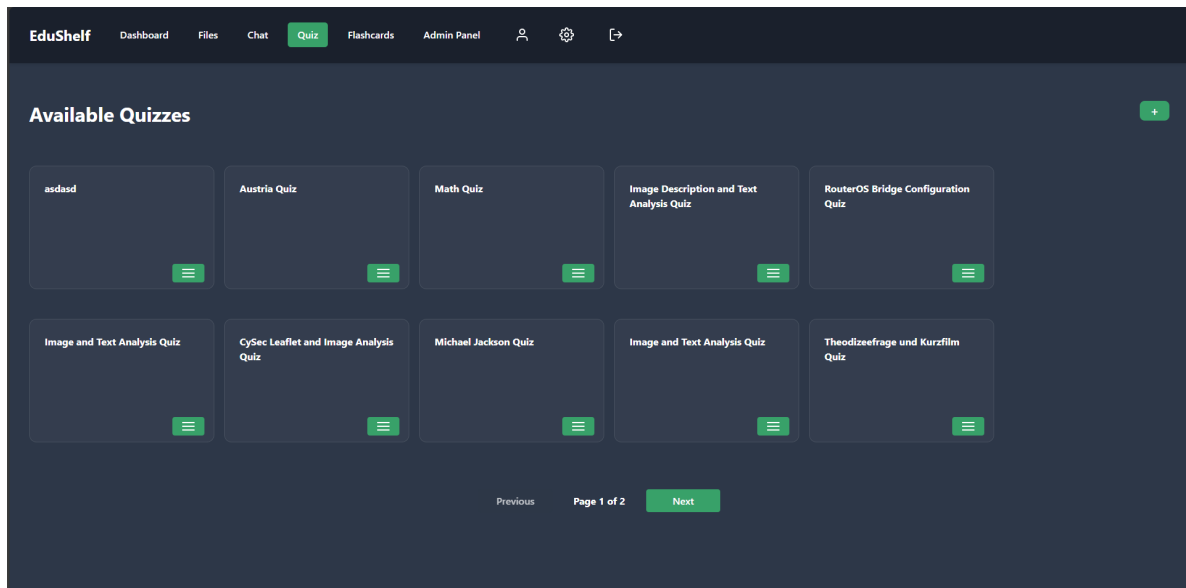


Abbildung 5.7: Übersicht aller verfügbaren Quizzes im Dark-Mode

5.12.1 Datenstruktur

Ein Quiz besteht aus einem Titel und einer Liste von Fragen. Jede Frage enthält wiederum eine Liste von Antworten, wobei eine oder mehrere als korrekt markiert sind. Diese hierarchische Struktur wird im State als verschachteltes Array-Objekt abgebildet:

```

1 const [questions, setQuestions] = useState([
2   {
3     text: '',
4     answers: [{ text: '', isCorrect: false }]
5   }
6 ]);

```

Listing 5.14: Datenstruktur eines Quiz

5.12.2 Ablauf eines Quiz

Der Zustand während eines laufenden Quizzes wird durch mehrere State-Variablen gesteuert:

- **currentQuestionIndex**: Index der aktuell angezeigten Frage
- **score**: Anzahl der bisher korrekt beantworteten Fragen
- **selectedAnswerId**: Um eine Mehrfachauswahl zu verhindern und visuelles Feedback zu geben
- **showScore**: Ein Boolean-Flag, das nach der letzten Frage auf **true** gesetzt wird, um die Ergebnisanzeige zu rendern

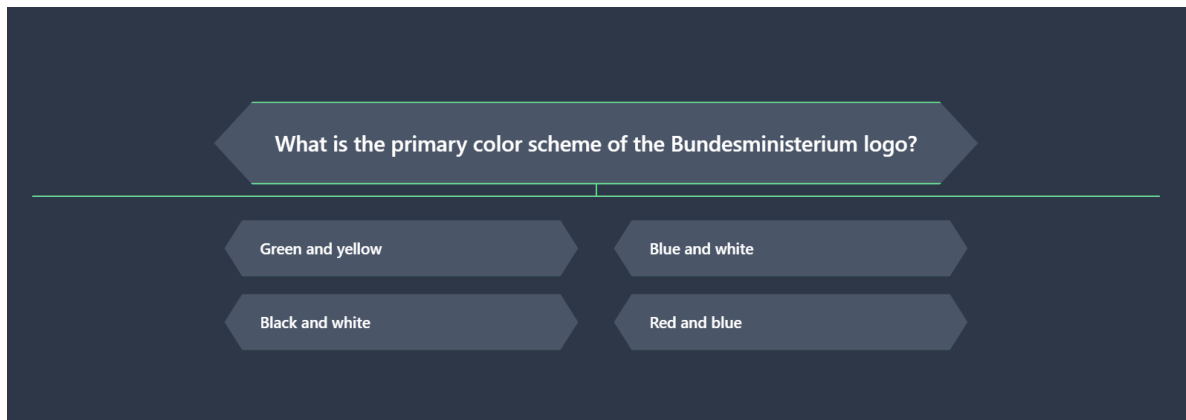


Abbildung 5.8: Quiz-Durchführung mit Antwort-Feedback im Dark-Mode

Das visuelle Feedback – Grün für richtig, Rot für falsch – wird durch CSS-Klassen gesteuert, die dynamisch basierend auf der Korrektheit der Antwort zugewiesen werden. Nach Beantwortung einer Frage wird **selectedAnswerId** gesetzt und alle Antwort-Buttons werden deaktiviert, um eine Mehrfachauswahl zu verhindern.

5.12.3 Quiz-Editor (QuizModal.jsx)

Die Erstellung und Bearbeitung von Quizzes erfolgt über `QuizModal.jsx`. Der Editor ermöglicht dynamisch das Hinzufügen und Entfernen von Fragen und Antworten.

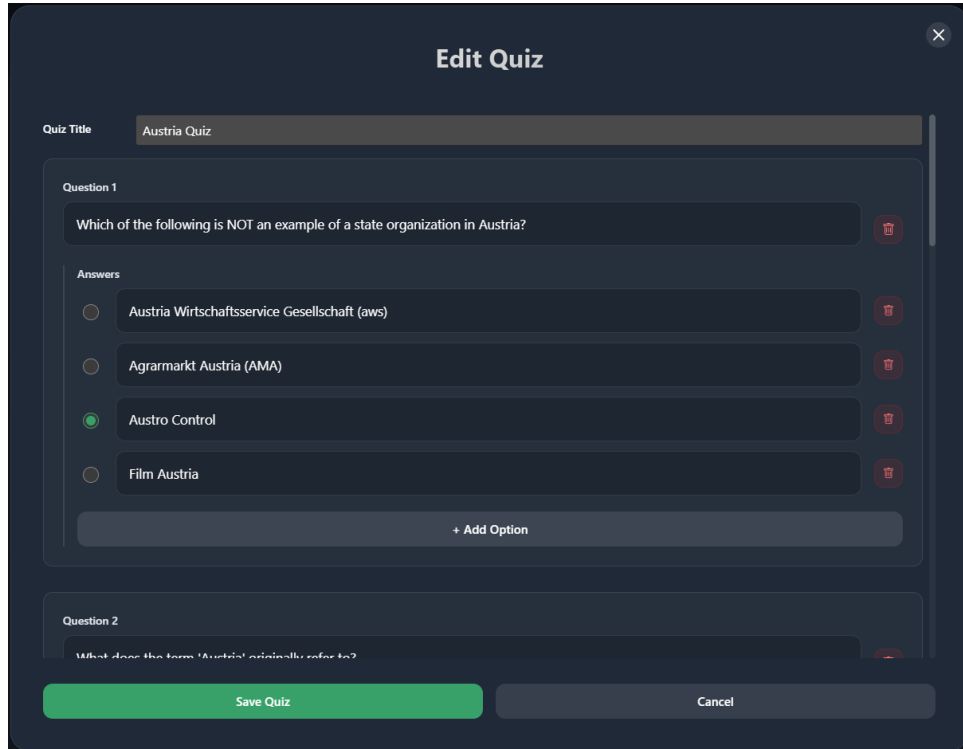


Abbildung 5.9: Quiz-Editor zum Erstellen und Bearbeiten von Quizzes

5.12.3.1 Dynamische Formular-Manipulation

Beim Ändern eines Antworttextes darf der State nicht direkt angepasst werden. Stattdessen muss eine Kopie der betroffenen Ebenen erstellt werden – das Prinzip der Immutability [9] in React:

```
1 const handleAnswerChange = (questionIndex, answerIndex, event) => {
2   const newQuestions = [...questions]; // Kopie des Arrays
3   newQuestions[questionIndex].answers[answerIndex].text
4     = event.target.value;
5   setQuestions(newQuestions); // Re-Render auslösen
6 };
```

Listing 5.15: Update-Logik für verschachtelte Antworten

Beim Speichern (`handleSubmit`) wird unterschieden, ob ein neues Quiz erstellt (POST) oder ein bestehendes bearbeitet (PUT) wird. Nach erfolgreicher API-Antwort ruft das Modal den übergebenen Callback `onQuizSaved()` auf, damit die Parentkomponente ihre Liste ohne erneuten Fetch aktualisiert.

5.13 Lernkarten (Flashcards.jsx)

Die Flashcard-Komponente ermöglicht klassisches Karteikarten-Lernen mit einem markanten 3D-Flip-Effekt. Sie demonstriert den gezielten Einsatz von CSS-Transformationen und einer unkonventionellen, aber effizienten State-Struktur.

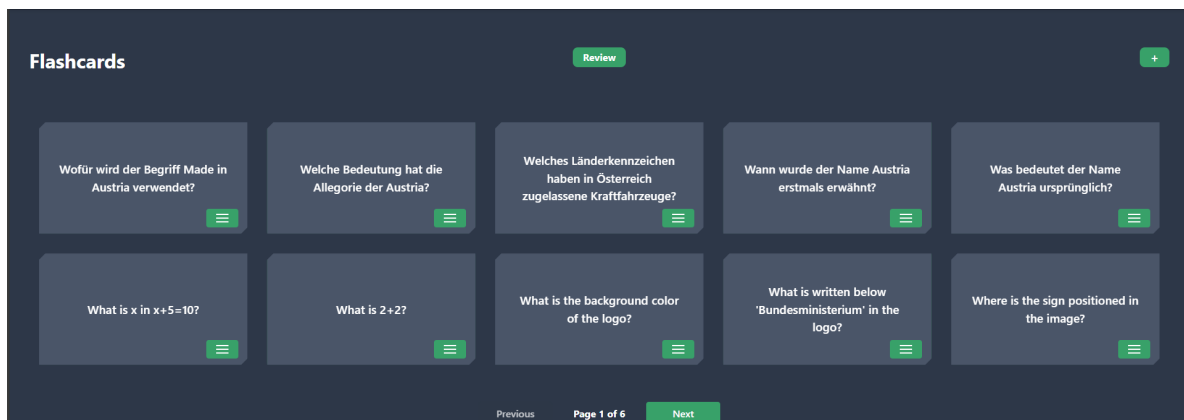


Abbildung 5.10: Lernkarten-Ansicht von EduShelf im Dark-Mode

5.13.1 CSS 3D-Flip-Effekt

Der realistische Dreh-Effekt wird ausschließlich über CSS 3D-Transformationen realisiert – ohne JavaScript-Animation. Das Schlüsselement ist die `perspective`-Eigenschaft, die dem Browser mitteilt, aus welcher Entfernung der 3D-Raum betrachtet wird:

```

1 .flashcard-scene {
2   perspective: 1000px;
3 }
4 .flashcard-container {
5   transition: transform 0.6s;
6   transform-style: preserve-3d;
7 }
8 .flashcard-scene.flipped .flashcard-container {
9   transform: rotateY(180deg);
10 }

```

Listing 5.16: CSS für den 3D-Flip-Effekt

Vor- und Rückseite der Karte sind übereinander gestapelt. Die Rückseite ist initial um 180° um die Y-Achse gedreht und somit unsichtbar. Durch Hinzufügen der CSS-Klasse `.flipped` dreht sich die gesamte Karte, was die Vorderseite verbirgt und die Rückseite sichtbar macht.

5.13.2 State-Verwaltung mit einem Set

Anstatt für jede Karte einen Boolean im Datenobjekt zu speichern, wird ein **Set** verwendet, das die IDs der umgedrehten Karten hält. Diese Trennung von UI-Zustand und Daten ist ein wichtiges Designprinzip:

```
1 const [flippedCards, setFlippedCards] = useState(new Set());
2
3 const flipCard = (id) => {
4   setFlippedCards(prevFlippedCards => {
5     const newFlippedCards = new Set(prevFlippedCards);
6     if (newFlippedCards.has(id)) {
7       newFlippedCards.delete(id);
8     } else {
9       newFlippedCards.add(id);
10    }
11    return newFlippedCards;
12  });
13 };
```

Listing 5.17: Nutzung eines Sets für UI-Zustand

Beim Rendern wird `flippedCards.has(card.id)` geprüft, um die CSS-Klasse `.flipped` bedingt hinzuzufügen. Diese Methode erlaubt es, beliebig viele Karten gleichzeitig umgedreht zu halten, ohne das Datenmodell zu verändern.

5.14 KI-Chat (Chat.jsx)

Die Chat-Komponente bietet eine Schnittstelle zu einem Large Language Model (LLM) und ist damit das KI-Herzstück von EduShelf. Sie erlaubt es, Lernmaterial direkt mit einem KI-Assistenten zu besprechen, Fragen zu stellen und Bilder als Kontext mitzuschicken.

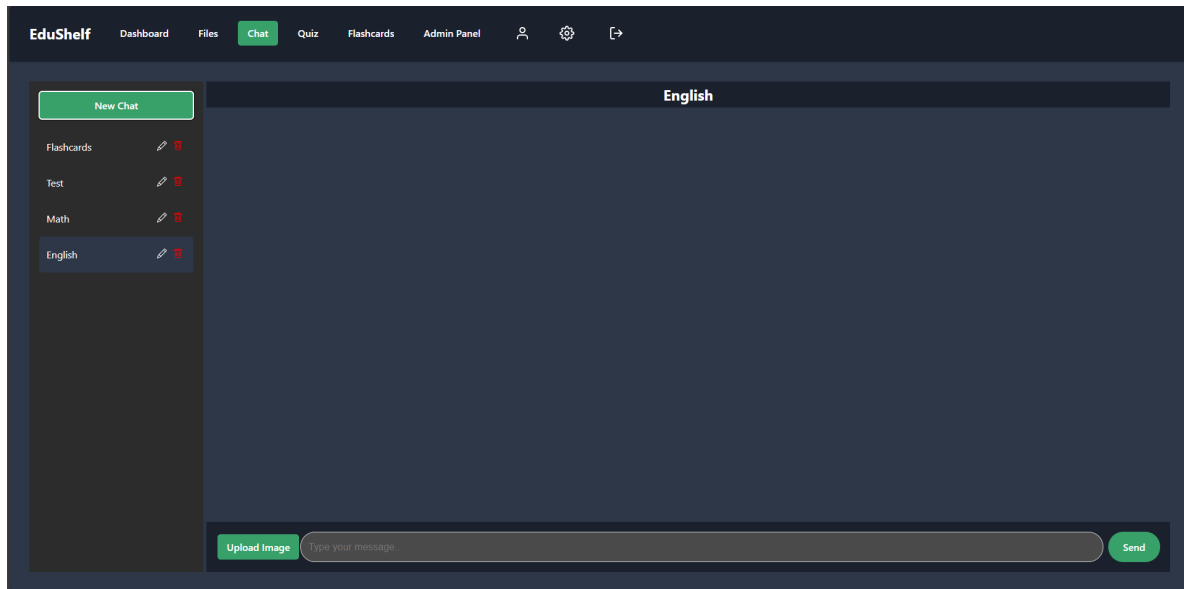


Abbildung 5.11: KI-Chat-Ansicht von EduShelf im Dark-Mode

5.14.1 Implementierungsdetails

5.14.1.1 Request/Response-Pattern

Die aktuelle Implementierung wartet auf die vollständige Antwort des Servers (Request/Response Pattern). Während des Wartens wird ein *Thinking Indicator* – drei animierte Punkte – über den `isLoading`-State eingeblendet. Der Eingabebereich wird in dieser Zeit deaktiviert, um doppelte Anfragen zu verhindern.

5.14.1.2 Markdown-Rendering

Da KI-Modelle häufig strukturierten Text mit Codeblöcken und Listen ausgeben, wird `react-markdown` in Kombination mit `rehype-highlight` verwendet. Diese Bibliotheken konvertieren die Markdown-Antwort sicher in HTML und bieten Syntax-Highlighting für Codeblöcke:

```
1 import ReactMarkdown from 'react-markdown';
2 import rehypeHighlight from 'rehype-highlight';
3
4 <ReactMarkdown rehypePlugins={[rehypeHighlight]}>
5   {message.content}
6 </ReactMarkdown>
```

Listing 5.18: Markdown-Rendering mit Syntax-Highlighting

5.14.1.3 Bildupload als Input

Benutzer können Bilder als Kontext an das Modell übermitteln. Das gewählte Bild wird mittels `URL.createObjectURL()` sofort lokal in einer Vorschau angezeigt, bevor es als Blob an den Server gesendet wird. Dies gibt dem Nutzer unmittelbares visuelles Feedback, ohne auf die Serverantwort warten zu müssen.

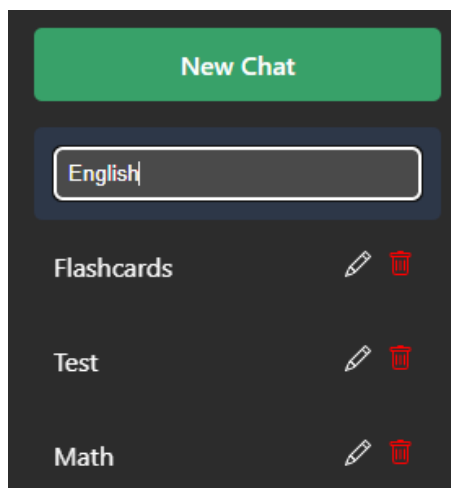
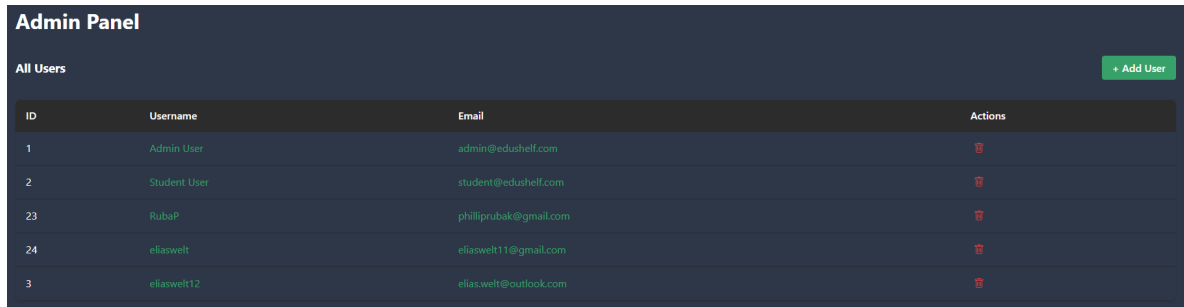


Abbildung 5.12: Chat-Eingabebereich mit Bildupload-Option

5.15 Admin Panel (AdminPanel.jsx)

Das `AdminPanel.jsx` ist das Kontrollzentrum für Systemadministratoren und demonstriert fortgeschrittene Patterns wie zweistufige Zugriffssicherung und kaskadierende Datenabfragen. Der Zugriff auf diese Route wird auf zwei Ebenen geschützt:



ID	Username	Email	Actions
1	Admin User	admin@edushelf.com	
2	Student User	student@edushelf.com	
23	RubaP	philliprubak@gmail.com	
24	eliaswelt	eliaswelt11@gmail.com	
3	eliaswelt12	elias.welt@outlook.com	

Abbildung 5.13: Admin-Panel von EduShelf im Dark-Mode

5.15.1 Zweistufige Zugriffssicherung (RBAC)

- **1. Client-Side Routing:** In `App.jsx` wird geprüft, ob das Benutzerobjekt die Rolle *Admin* enthält. Falls nicht, wird der Nutzer sofort umgeleitet.
- **2. Server-Side API:** Selbst wenn ein Nutzer die UI umgehen würde, lehnt das Backend Anfragen an admin-spezifische Endpunkte ab.

Diese zweistufige Strategie folgt dem Prinzip *Defense in Depth*: Die clientseitige Prüfung verbessert die UX (keine sichtbaren Admin-Menüpunkte für normale Nutzer), während die serverseitige Prüfung die tatsächliche Sicherheitsbarriere darstellt.

5.15.2 Kaskadierende Datenabfragen (Master-Detail)

Das Panel ermöglicht CRUD-Operationen für Benutzer. Besonders hervorzuheben ist die verschachtelte Datenansicht: Klickt ein Admin auf einen Benutzer, werden dessen Dateien asynchron nachgeladen.

```
1 const handleClick = (user) => {
2   setSelectedUser(user);
3   setEditUserMode(false);
4   // Dateien werden erst beim Klick geladen (Lazy Loading)
5   fetchUserFiles(user.userId);
6 };
7
8 const fetchUserFiles = async (userId) => {
9   try {
10    setLoading(true);
11    const files = await api.get(`/Users/${userId}/documents`);
12    setUserFiles(files);
13  } catch (err) {
14    setError('Could not load user files.');
```

Listing 5.19: Loading von Detaildaten erst beim Nutzerklick

Dieses Vorgehen sorgt für eine schnelle initiale Ladezeit der Benutzertabelle, da die Dateimetadaten erst bei Bedarf abgerufen werden und nicht pauschal für alle Benutzer auf einmal.

5.16 Benutzerprofil (Profile.jsx)

Die Profilseite gibt dem Nutzer einen Überblick über seine Account-Informationen und ermöglicht grundlegende Kontoverwaltung. Diese Ansicht stellt ein einfaches, aber effektives Muster zum Anzeigen und Aktualisieren von Benutzerdaten dar und ist eine weitere Demonstration der durchgängig einfachen Bedienung und hohen UX von EduShelf

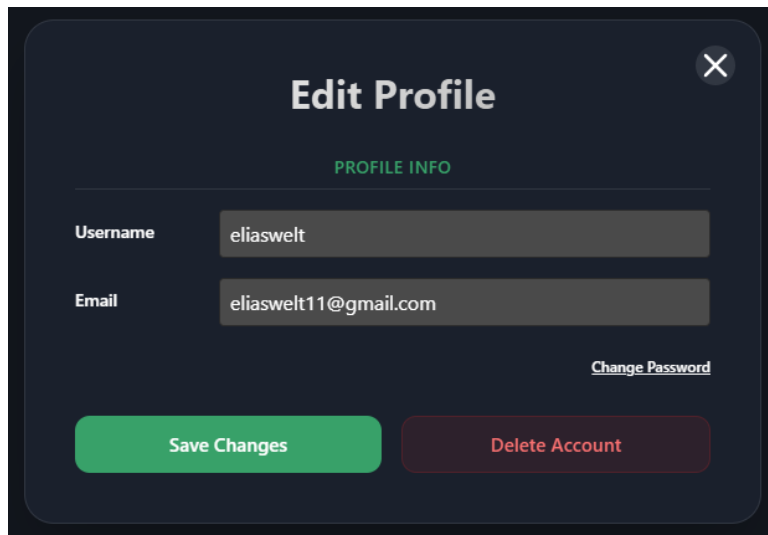


Abbildung 5.14: Profilseite von EduShelf im Dark-Mode

Die Profildaten werden beim Laden der Komponente aus dem `localStorage` entnommen – ohne zusätzlichen API-Aufruf. Da die Benutzerdaten (Name, E-Mail, Rollen) bereits beim Login abgerufen und lokal gespeichert wurden, stehen sie sofort zur Verfügung. Dies vermeidet einen unnötigen Netzwerkaufruf und sorgt für eine schnelle Darstellung der Seite.

5.17 Einstellungen und Benutzerpräferenzen (Settings.jsx)

Die Einstellungskomponente `Settings.jsx` ermöglicht dem Nutzer die individuelle Anpassung der Anwendungsdarstellung. Sie demonstriert ein durchgängiges Muster zur persistenten Speicherung von UI-Präferenzen im Browser.

5.17.1 Präferenz-Initialisierung beim Laden

Beim Starten der Komponente werden alle Benutzereinstellungen aus dem `localStorage` [6] geladen und als initialer State gesetzt. Dieses Vorgehen stellt sicher, dass die Einstellungen über Seitenreloads und Browsersitzungen hinweg erhalten bleiben:

```
1 useEffect(() => {  
2   const currentTheme = localStorage.getItem('theme');  
3   if (currentTheme === 'light') setLightMode(true);  
4  
5   const storedAnimations = localStorage.getItem('animationsEnabled');  
6   if (storedAnimations !== null)  
7     setAnimationsEnabled(storedAnimations === 'true');  
8  
9   const storedCompact = localStorage.getItem('compactMode');  
10  if (storedCompact !== null)  
11    setCompactMode(storedCompact === 'true');  
12 }, []);
```

Listing 5.20: Laden gespeicherter Präferenzen

Der Vergleich `storedValue !== null` ist dabei bewusst gewählt: Er unterscheidet zwischen einem nicht vorhandenen Eintrag (erster Aufruf – Standard gilt) und einem explizit auf `false` gesetzten Eintrag (Nutzer hat ausgeschaltet).

5.17.2 Theme-Umschaltung

Die Umschaltung zwischen Dark- und Light-Mode erfolgt durch direktes Manipulieren des `data-theme`-Attributs am `<html>`-Element. Dieses Attribut ist der Ankerpunkt für das CSS-Variablen-Theming:

```
1 const handleLightModeToggle = () => {  
2   const newLightMode = !lightMode;  
3   setLightMode(newLightMode);  
4   if (newLightMode) {  
5     document.documentElement.setAttribute('data-theme', 'light');  
6     localStorage.setItem('theme', 'light');  
7   } else {  
8     document.documentElement.removeAttribute('data-theme');  
9     localStorage.removeItem('theme');  
10  }  
11 };
```

Listing 5.21: Direkte DOM-Manipulation zur Theme-Umschaltung

Die Entfernung des Attributs (`removeAttribute`) statt setzen auf einen leeren String ist bewusst gewählt: Ohne das Attribut greift das Standard-CSS (`:root`) – also der Dark-Mode – und es existiert kein leerer Attributwert im DOM.

5.17.3 Schließen der Einstellungsseite

Die Einstellungsseite ist als Modal-Route realisiert. Das Schließen erfolgt nicht durch einen hartcoded Redirect, sondern durch `navigate(-1)` – den Zurück-Schritt in der Browser-Historie. Dadurch gelangt der Nutzer immer zur zuletzt besuchten Seite zurück, unabhängig davon, von wo aus die Einstellungen geöffnet wurden.

5.18 Benutzeroberfläche und Design (UI/UX)

5.18.1 Styling-Strategie

Das Projekt verwendet natives CSS mit CSS-Variablen (Custom Properties) [10] für das Theming. Dies bietet maximale Flexibilität ohne den Overhead großer Frameworks. Perfekt für unser eigen gewähltes Farbschema Globale Styles sind in `index.css` definiert, während komponentenspezifische Styles (z. B. `Chat.css`) lokal importiert werden.

5.18.2 Dark Mode Implementierung

-heutzutage ist der Dark Mode ist nicht nur eine kosmetische Spielerei, sondern eine zentrale Anforderung der Nutzerinnen und Nutzer. Deswegen wurde dieser natürlich auch bei EduShelf umgesetzt. Er wurde über CSS-Variablen realisiert, die je nach Attribut am HTML-Tag ihre Werte ändern.

```
1 :root {
2   /* Standard (Dark Mode) */
3   --primary-color: #222831;
4   --text-color: #FFFFFF;
5 }
6
7 [data-theme='light'] {
8   /* Light Mode Overrides */
9   --primary-color: #FFFFFF;
10  --text-color: #222831;
11 }
12
13 body {
14   background-color: var(--primary-color);
15   color: var(--text-color);
16 }
```

Listing 5.22: Theme-Switching mittels CSS-Variablen

Ein `useEffect`-Hook in `App.jsx` prüft beim Start den `localStorage` [6], ob der Nutzer eine Präferenz gespeichert hat, und setzt das Attribut entsprechend. Dies verhindert den Flash of Inaccurate Color Theme".

5.19 Detaillierte Analyse ausgewählter Module

Um die technische Tiefe der Implementierung zu verdeutlichen, werden im Folgenden drei Kernmodule detailliert anhand ihres Quellcodes analysiert.

5.19.1 Quiz-Editor Logik (QuizModal.jsx)

Der Quiz-Editor ermöglicht das Erstellen und Bearbeiten von verschachtelten Datenstrukturen (Quiz -> Fragen -> Antworten). Die Herausforderung hierbei ist das State-Management einer mehrdimensionalen Struktur.

5.19.1.1 Zustandsverwaltung

Der State `questions` ist ein Array von Objekten. Jedes Objekt repräsentiert eine Frage und enthält wiederum ein Array von `answers`.

```
1 const [questions, setQuestions] = useState([  
2   {  
3     text: '',  
4     answers: [{ text: '', isCorrect: false }]  
5   }  
6 ]);
```

Listing 5.23: Initialer State im QuizModal

5.19.1.2 Dynamische Formular-Manipulation

Beim Ändern eines Antworttextes darf der State nicht direkt angepasst werden. Stattdessen muss eine Deep Copy der betroffenen Ebenen erstellt werden. `handleAnswerChange` demonstriert dabei das Prinzip Immutability [9] von React:

```
1 const handleAnswerChange = (questionIndex, answerIndex, event) => {  
2   const newQuestions = [...questions];  
3   newQuestions[questionIndex].answers[answerIndex].text = event.target.  
4     value;  
5   setQuestions(newQuestions);  
6 };
```

Listing 5.24: Update-Logik für verschachtelte Antworten

5.19.1.3 Validierung und Speichern

Beim Speichern (`handleSubmit`) werden die Daten an die API gesendet. Hierbei wird unterschieden, ob ein neues Quiz erstellt (`POST`) oder ein bestehendes bearbeitet (`PUT`) wird.

5.19.2 Flashcards: 3D-Interaktion und State

Die Komponente `Flashcards.jsx` verwendet eine interessante Datenstruktur für den Zustand der umgedrehten Karten. Anstatt für jede Karte einen Boolean im Objekt zu speichern (was eine Änderung des Datenmodells erfordern würde), wird ein `Set` verwendet, das die IDs der umgedrehten Karten speichert.

```
1 const [flippedCards, setFlippedCards] = useState(new Set());  
2  
3 const flipCard = (id) => {  
4     setFlippedCards(prevFlippedCards => {  
5         const newFlippedCards = new Set(prevFlippedCards);  
6         if (newFlippedCards.has(id)) {  
7             newFlippedCards.delete(id);  
8         } else {  
9             newFlippedCards.add(id);  
10        }  
11        return newFlippedCards;  
12    });  
13 };
```

Listing 5.25: Nutzung eines Sets für UI-State

Diese Methode ist performant und trennt den UI-Zustand (ist die Karte gedreht?) sauber von den eigentlichen Daten (Frage/Antwort). Beim Rendern wird `flippedCards.has(card.id)` geprüft, um die CSS-Klasse `.flipped` bedingt hinzuzufügen.

5.19.3 Admin Panel: Kaskadierende Datenabfragen

Im `AdminPanel.jsx` muss beim Klick auf einen Benutzer dessen Dateiliste geladen werden. Dies ist ein klassisches Master-DetailSzenario. Um unnötigen Netzwerktraffic zu vermeiden, werden die Dateien nicht initial für alle Benutzer geladen (Lazy Loading").

```
1 const handleClick = (user) => {  
2     setSelectedUser(user);  
3     setError('');  
4     setEditUserMode(false);  
5     fetchUserFiles(user.userId);  
6 };  
7  
8 const fetchUserFiles = async (userId) => {  
9     try {  
10         setLoading(true);  
11         const files = await api.get(`/Users/${userId}/documents`);  
12         setUserFiles(files);  
13     } catch (err) {  
14         setError('Could not load user files.');15     } finally {  
16         setLoading(false);  
17     }  
18 };
```

Listing 5.26: Asynchrones Laden von Detaildaten

Diese Implementierung sorgt für eine schnelle initiale Ladezeit der Benutzertabelle, da die (potenziell große) Menge an Dateimetadaten erst bei Bedarf (Ön-Demand") abgerufen wird.

5.20 Deployment und Betrieb

Für den späteren Einsatz wird die Frontend-Anwendung containerisiert. Dies gewährleistet eine konsistente Laufzeitumgebung unabhängig vom Host-System.

5.20.1 Docker-Containerisierung

Es wird ein „Multi-Stage Build“ Prozess verwendet [11], um das finale Image so klein wie möglich zu halten.

1. **Build-Stage:** Ein Node.js Image wird verwendet, um die Abhängigkeiten zu installieren und den Build-Prozess (`npm run build`) [2] ausführen. Das Ergebnis ist ein Ordner `dist`, der nur die statischen Dateien (HTML, CSS, JS) enthält.
2. **Production-Stage:** Ein leichtgewichtiger Nginx-Webserver wird als Basis genommen. Die Dateien aus der Build-Stage werden in das Verzeichnis `/usr/share/nginx/html` kopiert.

```
1 # Stage 1: Build
2 FROM node:20-alpine as build
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci
6 COPY . .
7 RUN npm run build
8
9 # Stage 2: Serve
10 FROM nginx:alpine
11 COPY --from=build /app/dist /usr/share/nginx/html
12 COPY nginx.conf /etc/nginx/conf.d/default.conf
13 EXPOSE 80
14 CMD ["nginx", "-g", "daemon off;"]
```

Listing 5.27: Dockerfile für das Frontend

5.20.2 Nginx Konfiguration

Da es sich um eine SPA handelt, muss der Webserver so konfiguriert werden, dass er für alle Routen (z.B. `/files`, `/chat`) immer die `index.html` ausliefert (Fallback"). Andernfalls würde ein direkter Aufruf einer Unterseite zu einem 404-Fehler führen, da diese Dateien physisch nicht existieren.

5.21 Herausforderungen und Lösungen

Während der Entwicklung traten verschiedene technische Herausforderungen auf, die spezifische Lösungen erforderten.

5.21.1 Synchronisation des UI-Status

Problem: Wenn ein Nutzer ein Quiz bearbeitet, muss die Liste der Quizzes im Hintergrund aktuell bleiben. **Lösung:** Callback-Funktionen (z.B. `onQuizSaved`) werden an die Modal-Komponenten übergeben. Nach erfolgreicher API-Antwort ruft das Modal diese Funktion auf, und die Elternkomponente aktualisiert ihren lokalen State, ohne einen erneuten Fetch auszuführen. Optimistic UI Updates wurden in Betracht gezogen, aber zugunsten der Datenkonsistenz verworfen.

5.21.2 Performance bei großen Dateilisten

Problem: Das Rendern von hunderten Dateien führte zu Verzögerungen. **Lösung:** Durch die Pagination lässt sich dies optimieren.

5.22 Ausblick und Erweiterungsmöglichkeiten

Das aktuelle Frontend bietet eine solide Basis, kann jedoch in Zukunft erweitert werden:

- **Automatisierte Tests:** Integration von `Vitest` für Unit-Tests der Logik und `Cypress` für End-to-End Tests der Benutzeroberfläche.
- **PWA (Progressive Web App):** Durch Hinzufügen eines Service Workers und eines Web App Manifests könnte die Anwendung offline-fähig gemacht und auf Mobilgeräten installiert werden.
- **Internationalisierung (i18n):** Nutzung von Bibliotheken wie `react-i18next`, um die Oberfläche in mehreren Sprachen anzubieten.

5.23 Fazit

Die Frontend-Implementierung von EduShelf demonstriert die Leistungsfähigkeit moderner Webtechnologien. Durch den Einsatz von React und einer komponentenbasierte Architektur entstand eine wartbare, skalierbare und benutzerfreundliche Anwendung. Die Integration komplexer Features wie Real-Time-Editoren, KI-Chat und interaktive Lernmodule in einer Single Page Application zeigt, dass Webanwendungen nativen Programmen in nichts nachstehen müssen.

Zeiterfassung

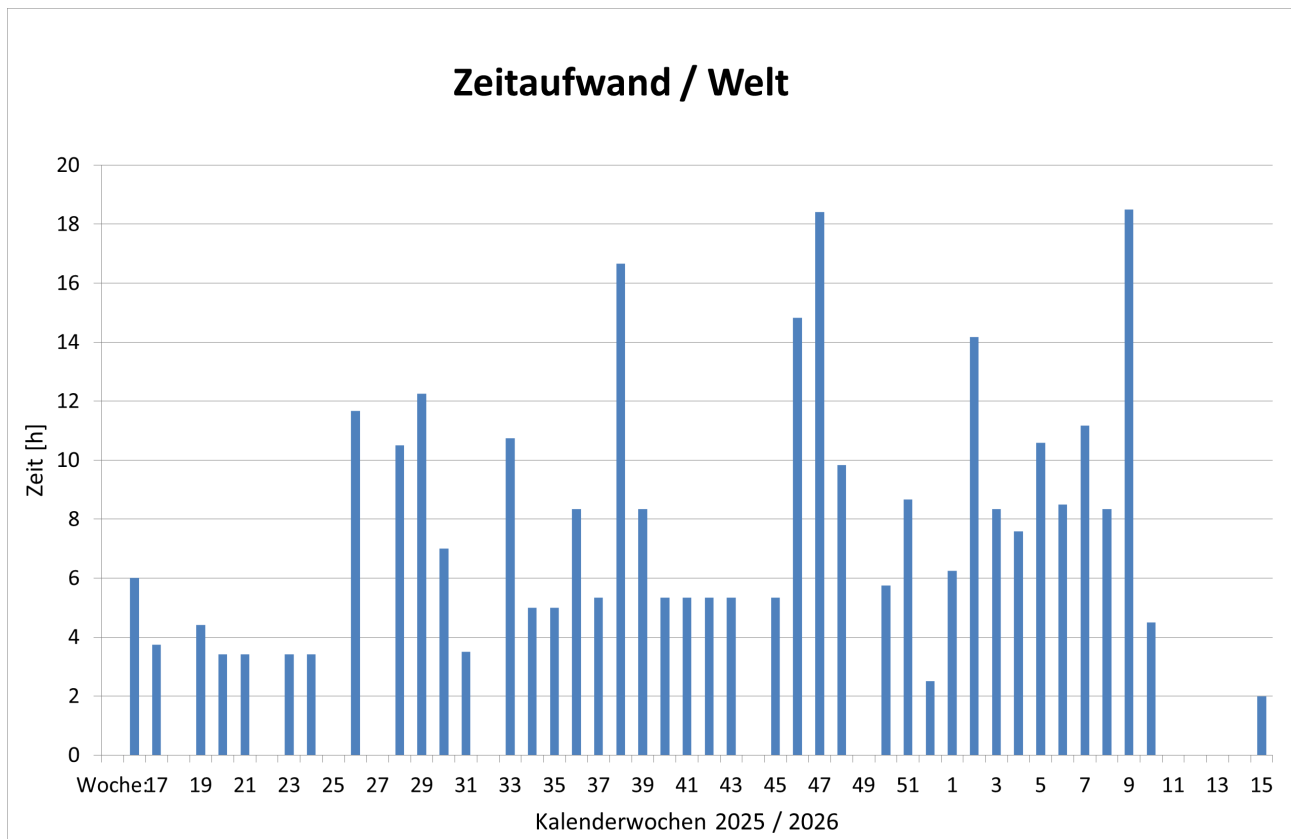


Abbildung 5.15: Frontend Zeitaufwand

Gesamt: 318,75h

Freizeit: 236,46h

6 Backend Implementierung

Die technische Umsetzung des EduShelf-Backends stellt das fundamentale Gerüst der Anwendung dar. Es wurde als ASP.NET Core Web API konzipiert, die als zentrale Schnittstelle zwischen dem Frontend, der Datenbank, dem KI-Modell und dem Dateispeicherdienst dient. Diese Entscheidung für eine kompakte Architektur wurde bewusst getroffen, um die Entwicklungskomplexität gering zu halten, während durch den Einsatz moderner .NET 8 Technologien gleichzeitig eine hohe Performance, Skalierbarkeit und Wartbarkeit gewährleistet wird.

In den folgenden Abschnitten wird die Architektur, der Technologie-Stack, das Datenmodell sowie die spezifischen Implementierungsdetails der Kernkomponenten detailliert dargestellt.

6.1 Systemarchitektur

Die Architektur des Systems folgt dem bewährten Muster einer *Layered Architecture* (Schichtenarchitektur) [12], um eine saubere Trennung der Verantwortlichkeiten (*Separation of Concerns*) zu erzielen. Dies erleichtert nicht nur die Testbarkeit mittels Unit- und Integrationstests, sondern ermöglicht auch zukünftige Erweiterungen ohne das Gesamtsystem neu schreiben zu müssen.

Die Anwendung gliedert sich in vier Hauptschichten:

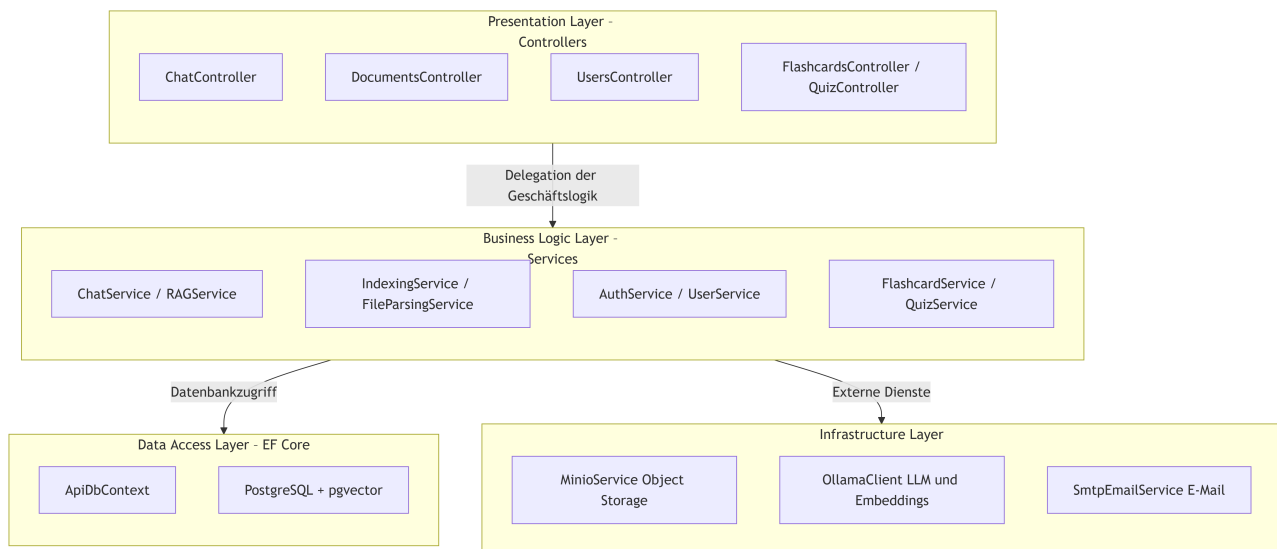


Abbildung 6.1: Schichtenarchitektur des EduShelf Backends

6.1.1 Presentation Layer (Controllers)

Diese oberste Schicht nimmt HTTP-Anfragen (Requests) entgegen und gibt HTTP-Antworten (Responses) zurück. Sie dient als Tor zur Außenwelt.

- **Verantwortlichkeit:** Routing, Deserialisierung von JSON-Body-Daten, Validierung von Eingaben und Formatierung der Ausgaben.
- **Validierung:** Mithilfe der Bibliothek *FluentValidation* werden eingehende Datenmodelle (DTOs) noch vor der Verarbeitung auf Korrektheit geprüft (z.B. `UserRegisterValidator`, der sicherstellt, dass E-Mail-Adressen gültig sind und Passwörter Mindestanforderungen erfüllen).
- **Controller:** Die Controller sind schlank gehalten (*Thin Controllers*) und delegieren die eigentliche Geschäftslogik sofort an die Service-Schicht. Beispiel: Der `ChatController` leitet die Benutzeranfrage direkt an den `ChatService` weiter.

6.1.2 Business Logic Layer (Services)

Hier residiert das „Gehirn“ der Anwendung. Alle Regeln und Prozesse sind in dieser Schicht implementiert.

- **Service-Klassen:** Jeder Bereich hat seinen eigenen Service (z.B. `AuthService`, `IndexingService`, `RAGService`).

- **Orchestrierung:** Services koordinieren den Datenfluss zwischen der Datenbank, externen Diensten (wie MinIO oder Ollama) und der Präsentationsschicht.
- **Dependency Injection:** Alle Services werden über den in .NET integrierten Dependency Injection (DI) Container registriert, was die Austauschbarkeit von Implementierungen zur Laufzeit ermöglicht.
- **Hintergrundverarbeitung:** Für langlaufende Aufgaben wie das Indizieren von Dokumenten wird eine `BackgroundJobQueue` eingesetzt. Dies entkoppelt rechenintensive Prozesse vom HTTP-Request-Zyklus und verhindert Timeouts im Frontend. Der `DocumentService` reiht Aufgaben ein, die dann von einem `BackgroundService` asynchron abgearbeitet werden.

6.1.3 Data Access Layer (Data)

Diese Schicht abstrahiert den Zugriff auf die persistente Datenspeicherung.

- **Entity Framework Core (EF Core):** Als Object-Relational Mapper (ORM) [13] ermöglicht EF Core die Definition des Datenmodells in C#-Klassen (siehe Abbildung 6.2). SQL-Queries werden zur Laufzeit generiert, was die Entwicklung beschleunigt und vor SQL-Injection schützt.
- **Repositories:** Ebenfalls wird das `DbContext`-Pattern direkt verwendet, was bei EF Core oft äquivalent zum Repository-Pattern bzw. Unit-of-Work-Pattern ist.

6.1.4 Infrastructure Layer

Diese Schicht beinhaltet die technische Anbindung an externe Systeme und Infrastrukturkomponenten.

- **MinIO:** Für die Speicherung unstrukturierter Daten (Blobs) wie PDF-Dokumente oder Bilder.
- **Ollama:** Schnittstelle zum lokalen KI-Modell.
- **E-Mail-Dienst:** Implementierung des `SmtpEmailService` zum Versenden von Bestätigungs-E-Mails.

6.2 Technologie-Stack

Das Backend setzt auf einen modernen, vollständig Open-Source-basierten Stack, der speziell für performante und KI-gestützte Anwendungen optimiert wurde.

- **ASP.NET Core 8:** Das zugrundeliegende Framework [14] bietet native Unterstützung für asynchrone Programmierung (`async/await`), was für I/O-lastige Operationen (Datenbank, Netzwerk) essenziell ist. Es ist plattformunabhängig und läuft in Linux-Containern.
- **PostgreSQL + pgvector:** PostgreSQL wurde aufgrund seiner Robustheit und Erweiterbarkeit gewählt. Die entscheidende Komponente für die RAG-Funktionalität ist die Erweiterung `pgvector` [15]. Sie ermöglicht das Speichern von hochdimensionalen Vektoren (Embeddings) direkt in der Datenbank und führt Ähnlichkeitssuchen (Nearest Neighbor Search) mittels Kosinus-Distanz oder Euklidischer Distanz performant durch.
- **Ollama:** Ein lokaler Server zur Ausführung von Large Language Models (LLMs) wie Llama 3 oder Qwen3. Durch die lokale Ausführung bleiben sensible Dokumentendaten auf dem Server und werden nicht an Drittanbieter-APIs (wie OpenAI) gesendet, was den Datenschutz massiv verbessert.
- **MinIO:** Ein S3-kompatibler Object Storage. Hochgeladene Dateien werden nicht als BLOB in der relationalen Datenbank gespeichert (was diese aufblähen würde), sondern effizient im Dateisystem über MinIO verwaltet.
- **Microsoft Semantic Kernel:** Ein SDK [16], das als Abstraktionsschicht für KI-Dienste dient. Es vereinfacht die Integration verschiedener KI-Modelle und bietet Mechanismen für Prompt-Templating, Memory-Management (Vektorspeicher-Abstraktion) und Function Calling.
- **PdfPig & Docnet.Core:** Spezialisierte Bibliotheken zur Extraktion von Text aus PDF-Dateien. Docnet.Core ist ein Wrapper um die native `pdfium`-Bibliothek und bietet extrem hohe Performance beim Rendern, während PdfPig reines C# ist und gut für Text-Extraktion geeignet ist.

6.3 Projekt-Setup und Konfiguration

Analog zum Frontend erfolgt der Einstieg in die Anwendung über eine definierte Startup-Logik, die in der `Program.cs` und den Konfigurationsdateien verankert ist.

6.3.1 Programm-Initialisierung (`Program.cs`)

Die `Program.cs` ist der Entry Point der ASP.NET Core Anwendung. Hier wird der „Generic Host“ gebaut, der Dependency Injection Container befüllt und die HTTP-Request-Pipeline (Middleware) konfiguriert. Die Middleware-Pipeline wird in folgender Reihenfolge konfiguriert: HTTPS-Redirection, CORS (`AllowWebApp`), `Authentication`, `Authorization` und `Controller-Mapping`. In der Entwicklungsumgebung werden zusätzlich Swagger und Swagger UI aktiviert. Alle applikationsspezifischen Services werden über die Erweiterungsmethode `AddApplicationServices` gesammelt im DI-Container registriert.

6.3.2 Konfiguration (`appsettings.json`)

Die Anwendungskonfiguration folgt dem *12-Factor App*-Prinzip [17]: Alle umgebungsabhängigen Parameter werden aus der Datei `appsettings.json` geladen und können zur Laufzeit durch Umgebungsvariablen überschrieben werden (z.B. in Docker Compose).

Die wichtigsten Konfigurationen im Überblick:

- **ConnectionStrings:** Verbindungsstring zur PostgreSQL-Datenbank (Host, Port, Datenbankname, Zugangsdaten).
- **AIService:** Zentrale KI-Konfiguration. `ChatModel` gibt das Sprachmodell an (z.B. `qwen3:14b`), `EmbeddingModel` das Modell für die Vektorerzeugung (`nomic-embed-text`) und `VisionModel` das Modell für die Bildanalyse. `ContextLengthLimit` (Standard: 4096 Token) begrenzt die Kontextfenstergröße. Die `Prompts`-Untersektion enthält die vollständigen System-Prompts für alle KI-gesteuerten Features.
- **Minio:** Verbindungsdaten zum S3-kompatiblen Object Storage (Endpoint, Zugangsdaten, Bucket-Name).
- **EmailSettings:** SMTP-Konfiguration für den Versand von Registrierungs-Bestätigungsmails.

- **Seeding:** Passwörter für vordefinierten Admin- und Student-Benutzer, die beim ersten Start automatisch angelegt werden.

6.4 Datenmodelle und Datenbankdesign

Das Datenmodell wurde entworfen, um sowohl relationale Benutzerdaten als auch unstrukturierte Dokumenteninhalte effizient zu verwalten. EF Core Migrationen stellen sicher, dass das Datenbankschema synchron zum Code bleibt.

6.4.1 Kernelemente

- **User:** Die zentrale Identitätseinheit. Enthält `Username`, `Email`, `PasswordHash` (gespeichert als BCrypt-Hash), sowie Felder für die E-Mail-Verifizierung (`IsEmailConfirmed`, `EmailConfirmationToken`).
- **Role & UserRole:** Implementierung einer Many-to-Many Beziehung für ein rollenbasiertes Zugriffssystem. Rollen sind z.B. „Student“ oder „Admin“.
- **Document:** Repräsentiert Metadaten eines hochgeladenen Dokuments. Das Feld `Path` verweist auf den Speicherort im MinIO-Bucket. Es besteht eine 1:n-Beziehung zum User.
- **DocumentShare:** Ermöglicht das Teilen von Dokumenten zwischen Benutzern (Many-to-Many via Join-Table). Diese Entität steuert den Zugriff für Nicht-Besitzer.
- **Cascading Deletes:** Die Datenbankkonfiguration (`OnModelCreating`) stellt sicher, dass beim Löschen eines Nutzers alle abhängigen Daten (Dokumente, Chats, Favoriten) automatisch entfernt werden, um Datenwaisen zu vermeiden.
- **DocumentChunk:** Dies ist die wichtigste Entität für die semantische Suche. Ein Dokument wird in viele kleine Abschnitte zerlegt.
 - **Content:** Der rohe Textinhalt des Abschnitts.
 - **Embedding:** Ein Vektor vom Typ `vector(768)`, der die semantische Bedeutung des Textes repräsentiert.

- **Page:** Referenz zur ursprünglichen Seite im Dokument (für Zitate).
- **ChatSession & ChatMessage:** Bilden den Verlauf einer Konversation ab. **ChatSession** gruppiert Nachrichten, während **ChatMessage** den eigentlichen Inhalt (User-Input oder AI-Response) speichert.

6.4.2 Entity Relationship Diagram (ERD)

Das folgende Diagramm visualisiert die Beziehungen zwischen den Entitäten. Zentral ist der **User**, der Dokumente besitzt und Lerninhalte (Flashcards, Quizzes) erstellt.

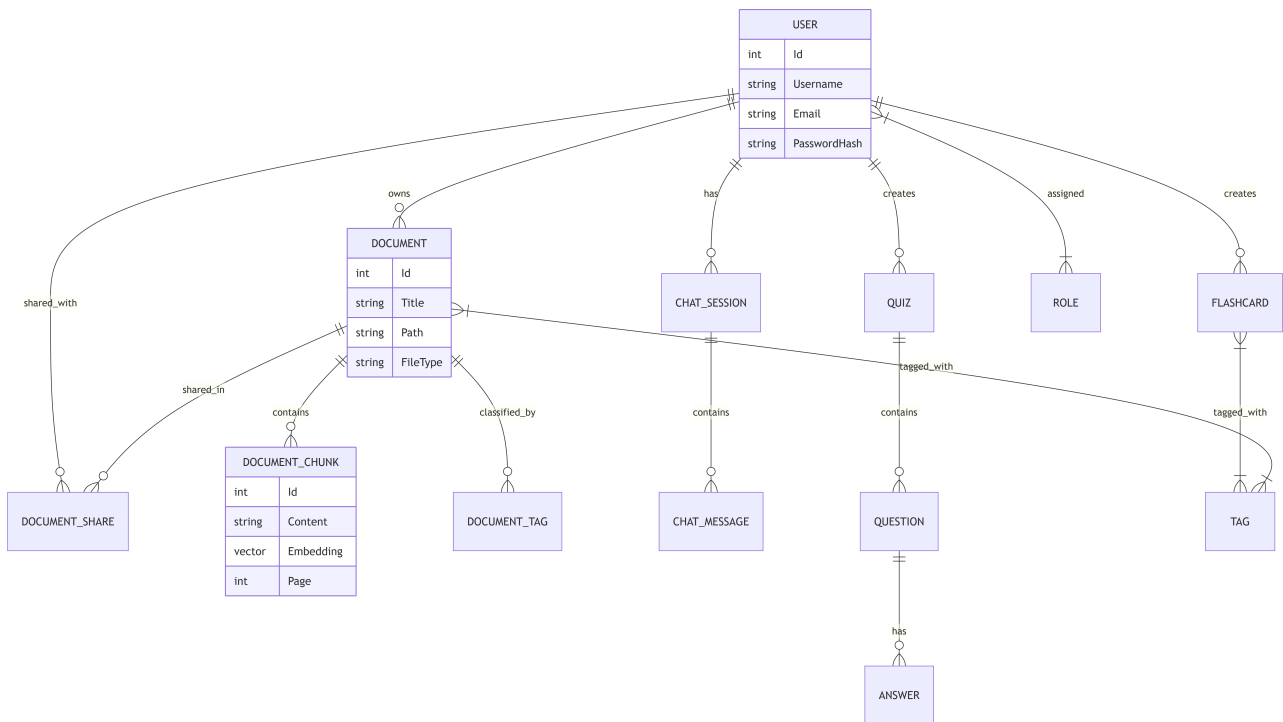


Abbildung 6.2: Entity Relationship Diagram des EduShelf Backends

6.5 API Design und Controller

Die Kommunikation mit dem Frontend erfolgt über eine RESTful API. Die acht Controller sind thematisch gruppiert und folgen REST-Konventionen (GET, POST, PUT, PATCH, DELETE). Nahezu alle Endpunkte sind hinter dem [Authorize]-Attribut geschützt, d.h. sie erfordern eine aktive Cookie-Session. Ausnahmen sind die öffentlichen Authentifizierungs-Endpunkte (Registrierung, Login, E-Mail-Bestätigung).

6.5.1 Ausgewählte API-Endpunkte

Methode	Route	Auth	Beschreibung
api/users			
POST	/	Anonym	Registrierung
POST	/login	Anonym	Login, setzt HttpOnly-Cookie
POST	/logout	Auth	Logout, löscht Session
GET	/me	Auth	Eigenes Profil abrufen
DELETE	/id	Admin	Nutzer löschen
api/documents			
GET	/	Auth	Eigene Dokumente (paginiert)
POST	/	Auth	Dokument hochladen (startet Indexierungs-Job)
DELETE	/id	Auth	Dokument inkl. MinIO-Datei löschen
GET	/download/id	Auth	Datei-Stream herunterladen
POST	/id/share	Auth	Dokument per E-Mail teilen
GET	/search	Auth	Volltext- und Tag-Suche
api/chat			
POST	/message	Auth	Nachricht senden, löst RAG-Pipeline aus
GET	/sessions	Auth	Chat-Sitzungen abrufen
POST	/sessions	Auth	Neue Chat-Sitzung erstellen
api/flashcards & api/quizzes			
POST	/flashcards/generate	Auth	Lernkarten per KI generieren
POST	/quizzes/generate	Auth	Quiz per KI generieren
api/tags			
GET	/	Auth	Alle Tags abrufen
POST	/	Admin	Neuen Tag erstellen

Tabelle 6.1: Wichtigste API-Endpunkte des EduShelf-Backends

6.6 Sicherheitskonzepte

6.6.1 Authentifizierung & Autorisierung

Die Authentifizierung erfolgt über Cookies (CookieAuthenticationDefaults).

- **Login-Prozess:** Der Nutzer sendet Anmeldedaten an POST /api/users/login. Der AuthService lädt den Benutzer inklusive seiner Rollen aus der Datenbank und überprüft den Passwort-Hash mittels BCrypt. Zusätzlich wird geprüft, ob die E-Mail-Adresse bereits bestätigt wurde – nicht-verifizierte Accounts können sich nicht einloggen.
- **Claims-Based Identity:** Bei erfolgreichem Login werden Claims (kryptographisch signierte Aussagen) über den Benutzer in das Cookie eingebettet: Benutzer-ID (ClaimTypes.NameIdentifier), E-Mail, Benutzername und alle zugewiesenen Rollen (ClaimTypes.Role). Diese Claims stehen serverseitig bei jedem Request über HttpContext.User zur Verfügung.
- **Session-Lebensdauer:** Das Cookie ist auf 120 Minuten befristet (IsPersistent = true, ExpiresUtc = +120 min).
- **Middleware:** Die Standard-ASP.NET Core Authentifizierungsmiddleware entschlüsselt das Cookie bei jedem Request und rekonstruiert das ClaimsPrincipal-Objekt.

```

1 var claims = new List<Claim>
2 {
3     new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()),
4     new Claim(ClaimTypes.Email, user.Email),
5     new Claim(ClaimTypes.Name, user.Username)
6 };
7 foreach (var userRole in user.UserRoles)
8     claims.Add(new Claim(ClaimTypes.Role, userRole.Role.Name));
9 var claimsIdentity = new ClaimsIdentity(
10     claims, CookieAuthenticationDefaults.AuthenticationScheme);
11 var authProperties = new AuthenticationProperties
12 {
13     IsPersistent = true,
14     ExpiresUtc = DateTimeOffset.UtcNow.AddMinutes(120)
15 };
16 await _httpContextAccessor.HttpContext.SignInAsync(
17     CookieAuthenticationDefaults.AuthenticationScheme,
18     new ClaimsPrincipal(claimsIdentity),
19     authProperties);

```

Listing 6.1: Claims-Aufbau beim Login in AuthService.cs

6.6.2 Rollenbasiertes Zugriffssystem (RBAC)

EduShelf implementiert ein einfaches, aber effektives Role-Based Access Control System. Rollen (**Student**, **Admin**) werden in der Tabelle **Roles** gespeichert und über die Join-Tabelle **UserRoles** Nutzern zugewiesen.

Jeder neu registrierte Nutzer erhält automatisch die Rolle **Student**. Die Rolle wird beim Erstellen des Nutzers aus der Datenbank geladen und dem **UserRoles**-Set hinzugefügt:

```
1 var studentRole = await _context.Roles
2   .SingleOrDefaultAsync(r => r.Name == Roles.Student);
3 user.UserRoles.Add(new UserRole { Role = studentRole });
```

Listing 6.2: Automatische Rollenzuweisung bei der Registrierung

In den Controllern wird die Autorisierung deklarativ über Attribute gesteuert:

- **[Authorize]** – Nur eingeloggte Nutzer (jede Rolle).
- **[Authorize(Roles = "Admin")]** – Nur Administratoren (z.B. für Nutzer-Löschung oder Tag-Verwaltung).

6.6.3 E-Mail-Verifizierung

Bei der Registrierung wird ein kryptographisch sicherer Bestätigungstoken (32 Byte, hexadezimal kodiert, 24 Stunden gültig) generiert und per E-Mail an den Nutzer gesendet. Schlägt der E-Mail-Versand fehl, wird der Nutzer-Datensatz sofort wieder gelöscht – das System verhält sich somit *atomar*. Bei Klick auf den Bestätigungsverlink führt der **AuthService** vier Prüfungen durch:

1. Nutzer mit dieser E-Mail existiert.
2. E-Mail ist noch nicht bestätigt.
3. Der übermittelte Token stimmt exakt mit dem gespeicherten überein.
4. Der Token ist nicht abgelaufen (**ExpiresAt** > **DateTime.UtcNow**).

Bei Erfolg werden **IsEmailConfirmed** = **true** gesetzt und Token sowie Ablaufzeit aus der Datenbank gelöscht. Zusätzlich werden alle eingehenden Daten mittels *FluentValidation* automatisch vor dem Controller-Aufruf validiert – ungültige Anfragen werden sofort mit HTTP 400 und einer strukturierten Fehlerliste abgelehnt.

6.6.4 Error Handling Middleware

Eine globale `ErrorHandlingMiddleware` fängt alle unbehandelten Ausnahmen ab. Dies verhindert, dass interne Implementierungsdetails (Stack Traces) an den Client durchsickern. Stattdessen werden standardisierte JSON-Fehlermeldungen und passende HTTP-Statuscodes zurückgegeben:

Exception-Typ	HTTP-Statuscode
<code>NotFoundException</code>	404 Not Found
<code>BadRequestException</code>	400 Bad Request
<code>UnauthorizedAccessException</code>	401 Unauthorized
<code>ValidationException</code>	400 Bad Request
<code>Sonstige Exception</code>	500 Internal Server Error

Tabelle 6.2: Mapping von Exceptions auf HTTP-Statuscodes

6.7 Ingestion-Pipeline (Datenverarbeitung)

Damit Dokumente durchsuchbar werden, müssen sie eine komplexe Verarbeitungspipeline durchlaufen. Dieser Prozess wird vom `IndexingService` gesteuert und ist rechenintensiv.

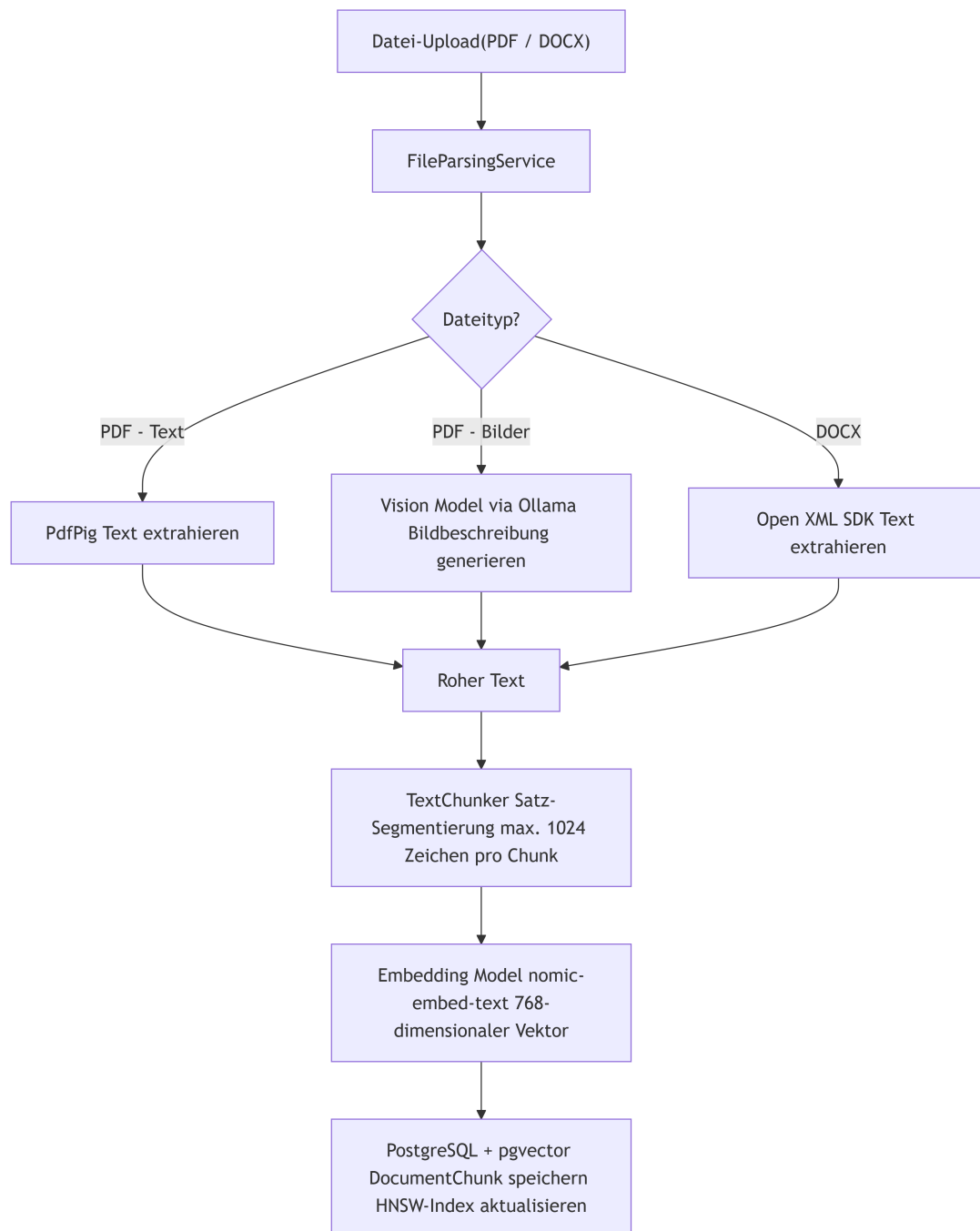


Abbildung 6.3: Ablaufdiagramm der Ingestion-Pipeline in EduShelf

6.7.1 Textextraktion (OCR & Parsing)

Der `FileParsingService` analysiert den Dateityp und wählt die passende Strategie:

- **PDF-Verarbeitung mit PdfPig:** Für PDF-Dateien wird die Bibliothek *UglyToad.PdfPig* verwendet. Sie erlaubt den Zugriff auf den rohen Textstream sowie die Extraktion von Bildern.
- **Hybride Bildanalyse:** Eingebettete Bilder werden isoliert und an ein lokales Vision-Model (via Ollama) gesendet. Der Prompt "Describe this image in detail" generiert eine textuelle Beschreibung, die in den Dokumentenindex einfließt. Dies ermöglicht die semantische Suche nach Bildinhalten (z.B. Diagramme, Graphen).
- **DOCX:** Nutzung des Open XML SDKs zum strukturierten Auslesen des `MainDocumentPart`.

6.7.2 Chunking (Intelligente Segmentierung)

LLMs haben ein begrenztes Kontextfenster, und die semantische Suche verliert bei zu großen Textblöcken an Präzision. Der `TextChunker` implementiert daher eine mehrstufige Strategie:

1. **Satz-Tokenisierung:** Der Text wird zunächst anhand von Satzzeichen (. ! ?) in einzelne Sätze zerlegt.
2. **Rekombination:** Diese Sätze werden iterativ zu einem Chunk zusammengefügt, bis eine konfigurierte Grenze von ca. 1024 Zeichen erreicht ist. Dies stellt sicher, dass semantische Zusammenhänge (Sätze) nicht willkürlich in der Mitte durchschnitten werden.
3. **Token-Limit:** Zusätzlich wird sichergestellt, dass die Chunks eine maximale Token-Anzahl (für das Embedding-Model) nicht überschreiten.

```

1 public static List<string> SplitBySentence(string text)
2 {
3     // Nutzung von Regex Lookbehind (?<=...), um das Satzzeichen zu behalten
4     var sentences = Regex.Split(text, @"(?<=[\.\!\?])\s+");
5     return new List<string>(sentences);
6 }

```

Listing 6.3: Implementierung der Satz-Segmentierung in `TextChunker.cs`

Das simple Splitten nach Zeichenanzahl würde Sätze zerreißen und den Kontext zerstören. Der hier gezeigte Ansatz garantiert semantische Integrität.

6.7.3 Embedding und Vektorspeicherung

Jeder Text-Chunk wird in einen hochdimensionalen Vektor umgewandelt und effizient gespeichert.

- **Embedding-Modell:** Nutzung von state-of-the-art Modellen wie *nomi-embed-text*, die semantische Ähnlichkeiten im Vektorraum abbilden.
- **HNSW Index:** In der PostgreSQL-Datenbank wird für die Spalte `Embedding` ein HNSW (Hierarchical Navigable Small World) Index [18] angelegt.

```
modelBuilder.Entity<DocumentChunk>()  
    .HasIndex(dc => dc.Embedding)  
    .HasMethod("hnsw")  
    .HasOperators("vector_12_ops");
```

Dieser Index ermöglicht eine extrem schnelle Nearest-Neighbor-Suche (ANN) auch bei Millionen von Vektoren, da nicht die gesamte Tabelle gescannt werden muss.

6.8 RAG (Retrieval-Augmented Generation) Workflow

Das Herzstück von EduShelf ist die RAG-Pipeline [19]: Anstatt das LLM allein auf sein allgemeines Wissen zu stützen, wird jede Benutzeranfrage zunächst dazu genutzt, relevante Textabschnitte aus den hochgeladenen Dokumenten abzurufen. Diese Abschnitte werden dann als *Kontext* in den Prompt eingefügt, bevor das LLM zur Antwort aufgefordert wird. Dies reduziert Halluzinationen und ermöglicht präzise sowie dokumentspezifische Antworten.

6.8.1 Systemübersicht und Datenfluss

Das folgende Diagramm visualisiert den vollständigen Anfrage-Antwort-Zyklus im RAG-System und zeigt, wie die verschiedenen Services zusammenarbeiten.

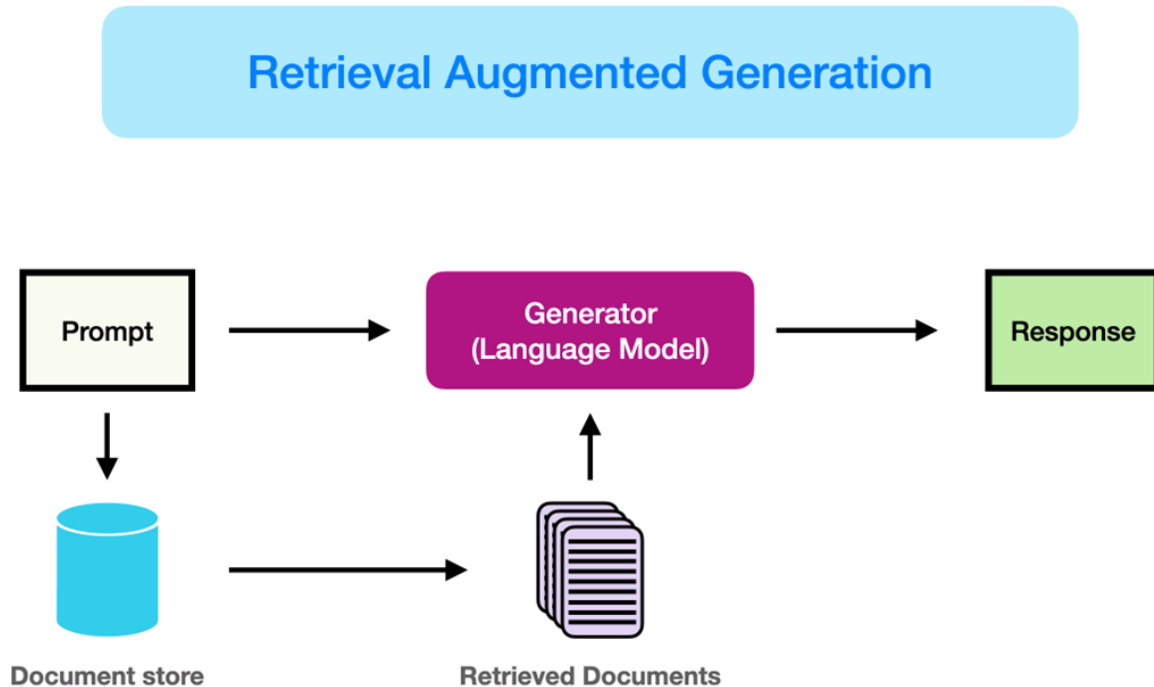


Abbildung 6.4: Architekturdiagramm des RAG-Systems in EduShelf (Quelle: [20])

Der Datenfluss bei einer Benutzeranfrage lässt sich in fünf klar abgegrenzte Schritte gliedern:

1. **Anfrage & Intent-Erkennung:** Der `ChatController` empfängt die Nachricht des Nutzers und leitet sie an den `ChatService` weiter. Dieser übergibt sie zuerst an den `IntentDetectionService`.
2. **Vektorielle Suche (Retrieval):** Basierend auf dem erkannten Intent ruft der `RetrievalService` die semantisch relevantesten `DocumentChunks` aus der PostgreSQL/pgvector-Datenbank ab.
3. **Kontext-Aufbereitung:** Der `PromptGenerationService` kombiniert den Konversationsverlauf, die abgerufenen Chunks und die aktuelle Frage zu einem Prompt.
4. **LLM-Inferenz:** Über das Microsoft Semantic Kernel SDK wird der Prompt an das lokal laufende Ollama-LLM gesendet.

5. **Antwort & Persistenz:** Die generierte Antwort wird in der Datenbank als `ChatMessage` gespeichert und an den Client zurückgegeben.

6.8.2 Schritt 1: Intent-Erkennung (IntentDetectionService)

Bevor eine semantische Suche stattfindet, muss das System verstehen, *was* der Nutzer möchte. Der `IntentDetectionService` sendet die Benutzeranfrage an das LLM mit einem speziellen System-Prompt, der das Modell anweist, eine strukturierte JSON-Antwort zu liefern:

```
1 var chatHistory = new ChatHistory();
2 chatHistory.AddSystemMessage(
3     _configuration.GetValue<string>("AIService:Prompts:Intent")
4     ?? "Identify the intent.");
5 chatHistory.AddUserMessage(userInput);
6
7 var result = await chatCompletionService.GetChatMessageContentAsync(
8     chatHistory);
9 var cleanedJson = JsonHelper.ExtractJson(result.Content ?? string.Empty);
10 var intent = JsonSerializer.Deserialize<Intent>(cleanedJson,
11     new JsonSerializerOptions { PropertyNameCaseInsensitive = true })
12     ?? new Intent { Type = "question", DocumentName = null };
```

Listing 6.4: Intent-Erkennung via LLM in `IntentDetectionService.cs`

Das `Intent`-Objekt enthält zwei Felder:

- **Type:** Klassifiziert die Anfrage als "question", "summarize", "flashcards" oder "quiz".
- **DocumentName:** Falls der Nutzer explizit ein Dokument nennt (z.B. „Erkläre mir Kapitel 3 aus Physik.pdf“), wird der Dateiname hier extrahiert.

Dieser Schritt ist entscheidend für den weiteren Prozess: Anstatt starr immer dieselbe RAG-Pipeline zu durchlaufen, wird die Antwort-Strategie dynamisch an den jeweiligen Kontext angepasst.

6.8.3 Schritt 2: Semantische Suche (RetrievalService)

Der `RetrievalService` ist für die Beschaffung der relevanten Dokumentenabschnitte zuständig. Er implementiert dabei eine zweistufige Strategie:

6.8.3.1 Primäre Suche: Name-basiertes Matching

Falls der Intent einen `DocumentName` enthält, sucht der Service direkt nach Chunks, deren übergeordnetes Dokument diesen Namen trägt. Dies erfolgt über einen PostgreSQL-spezifischen, groß-/kleinschreibungsunabhängigen Vergleich (`ILike`):

```

1 var chunks = await _context.DocumentChunks
2   .Include(dc => dc.Document)
3   .Where(dc => dc.Document.UserId == userId &&
4     (EF.Functions.ILike(dc.Document.Title, intent.DocumentName) ||
5     EF.Functions.ILike(dc.Document.Title,
6     documentNameWithoutExtension)))
7   .ToListAsync();

```

Listing 6.5: Dokumenten-spezifische Suche in RetrievalService.cs

6.8.3.2 Fallback: Semantische Vektorsuche

Wenn kein Dokumentname angegeben wurde oder keine Treffer gefunden wurden, wird eine semantische Suche über alle Chunks des Nutzers durchgeführt. Die Anfrage wird in einen Embedding-Vektor umgewandelt und mit allen gespeicherten Chunk-Vektoren per L2-Distanz verglichen:

```

1 var promptEmbedding = await _embeddingGenerator.GenerateAsync(userInput);
2 var k = GetDynamicK(userInput); // 5, 10 oder 20 Chunks
3
4 return await _context.DocumentChunks
5   .Include(dc => dc.Document)
6   .Where(dc => dc.Document.UserId == userId)
7   .OrderBy(dc => dc.Embedding.L2Distance(new Vector(promptEmbedding.Vector
8   )))
9   .Take(k)
   .ToListAsync();

```

Listing 6.6: Embedding-basierte Vektorsuche in RetrievalService.cs

Der Wert k (Anzahl der zurückgegebenen Chunks) ist dynamisch: Bei kurzen Anfragen (unter 10 Wörtern) werden $k = 5$, bei mittleren $k = 10$ und bei komplexen Anfragen (über 30 Wörter) $k = 20$ Chunks abgerufen.

6.8.4 Schritt 3: Prompt-Aufbau (PromptGenerationService)

Der PromptGenerationService baut eine vollständige ChatHistory auf, die drei Elemente enthält:

- Eine **System-Instruktion**, die das Verhalten des LLMs grundlegend steuert.
- Den **bisherigen Konversationsverlauf** der aktuellen Chat-Sitzung (für kontextsensitive Folgefragen).
- Die **aktuelle Benutzeranfrage**, angereichert mit dem abgerufenen Dokumentenkontext.

```
1 // Kontext aus Chunks zusammenstellen und auf Token-Limit kuerzen
2 foreach (var chunk in relevantChunks) {
3     contextText.AppendLine($"[{chunk.Document.Title}] {chunk.Content}");
4 }
5 contextString = _tokenService.Truncate(contextString, contextLengthLimit);
6
7 // Finale Benutzernachricht mit eingebettetem Kontext
8 finalUserMessage.AppendLine($"[Context from Documents]:\n{contextString}\n")
9 ;
10 finalUserMessage.AppendLine($"[User Question]: {userInput}");
11 chatHistory.AddUserMessage(finalUserMessage.ToString());
```

Listing 6.7: Prompt-Konstruktion in PromptGenerationService.cs

Ein zentrales Element ist die **Token-Limitierung** durch den `ITokenService`. Um das Kontextfenster des LLMs (Standardmäßig: 4096 Tokens) nicht zu überschreiten, werden die Chunks vor der Übergabe tokenisiert und präzise abgeschnitten (*Smart Truncation*). Dies verhindert API-Fehler und stellt sicher, dass die semantisch nächsten Chunks priorisiert werden.

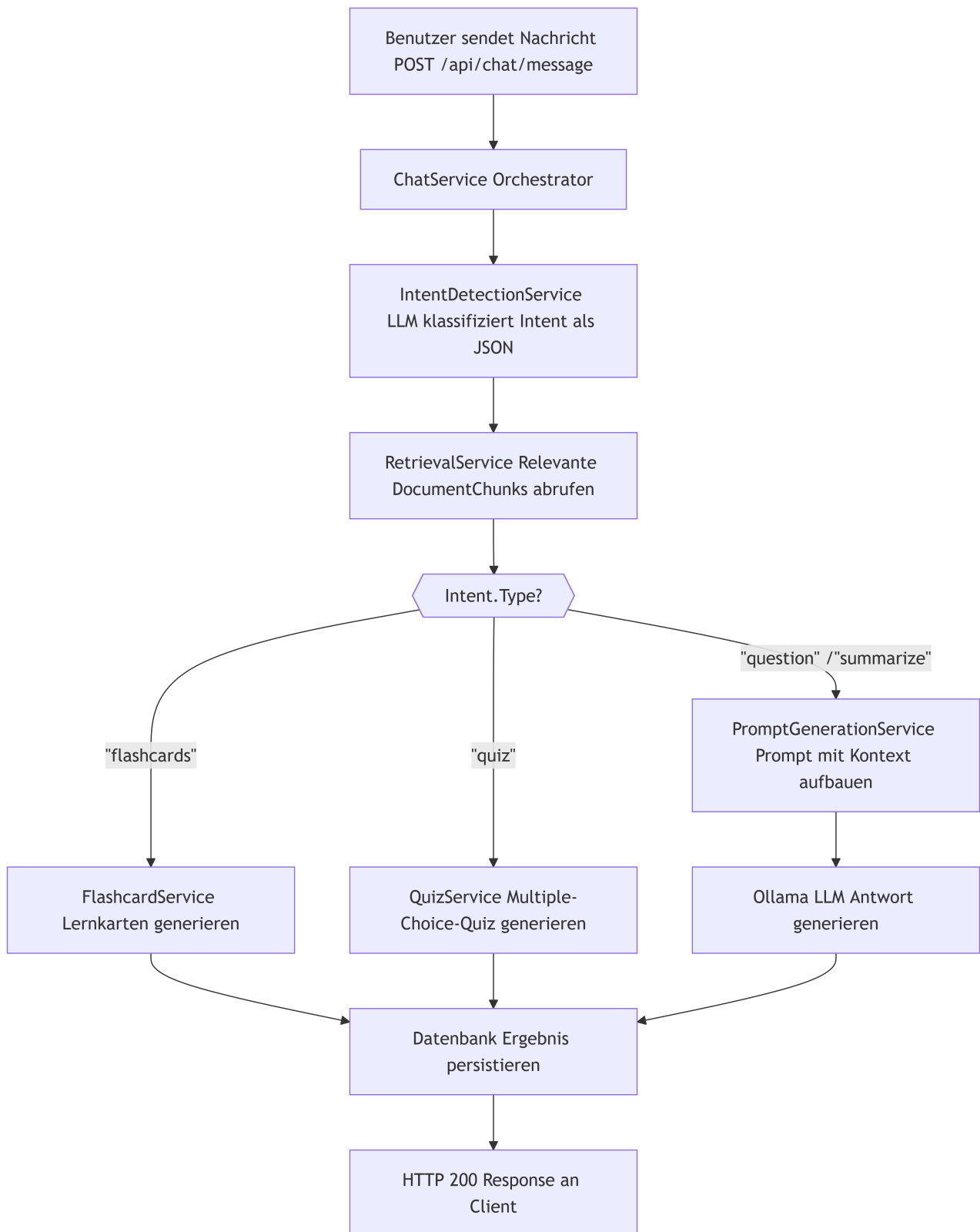


Abbildung 6.5: Intent-basiertes Branching im ChatService

6.8.5 Schritt 4: Orchestrierung und Intent-basiertes Branching (ChatService)

Der ChatService fungiert als zentraler Orchestrator und trifft nach der Retrieval-Phase eine **Intent-basierte Fallunterscheidung**. Dies unterscheidet EduShelf von einem einfachen Chatbot:

```

1 var intent = await _intentDetectionService.GetIntentAsync(promptInput);
2 var relevantChunks = await _retrievalService.GetRelevantChunksAsync(
3     promptInput, userId, intent);
4
5 if (intent.Type == "flashcards") {
6     await _flashcardService.GenerateFlashcardsAsync(
7         new GenerateFlashcardsRequest { Context = context, Count = 5 },
8         userId);
9     responseContent = "I have generated flashcards.";
10 }
11 else if (intent.Type == "quiz") {
12     await _quizService.GenerateQuizAsync(
13         new GenerateQuizRequest { Context = context, Count = 5 }, userId);
14     responseContent = "I have generated a quiz.";
15 }
16 else { // "question" oder "summarize"
17     var chatHistory = _promptGenerationService.BuildChatHistory(
18         chatSession, relevantChunks, promptInput);
19     var result = await chatCompletionService.GetChatMessageContentAsync(
20         chatHistory);
21     responseContent = result.Content ?? "Sorry, no response generated.";
22 }

```

Listing 6.8: Intent-basiertes Branching in ChatService.cs

Dieses Design ermöglicht es, über eine einzige Chat-Schnittstelle drei fundamentale Lernfunktionen anzubieten: konversationelles Frage-Antwort, automatische Flashcard-Generierung und automatische Quiz-Erstellung, alles gesteuert durch die Intention hinter der sprachlichen Eingabe des Nutzers.

6.8.6 Zusammenfassung: Retrieval-Augmented Generation [19]

Die Implementierung des RAG-Systems stellt sicher, dass das LLM stets auf den spezifischen Kontext der hochgeladenen Dokumente des Nutzers antwortet. Die wichtigsten Designprinzipien sind:

- **Datenprivacy:** Durch den lokalen Ollama-Server verlassen keine Dokumenteninhalte den Server.
- **Präzision:** Durch semantische Vektorsuche und optionales Name-Matching werden nur die relevantesten Chunks an das LLM übergeben.
- **Robustheit:** Token-basierte Kontextlimitierung verhindert LLM-Abstürze durch überlange Prompts.
- **Erweiterbarkeit:** Das Intent-Branching erlaubt es, neue KI-gestützte Features durch Hinzufügen weiterer Intent-Typen zu integrieren, ohne die Pipeline zu ändern.

6.9 KI-gestützte Lerninhaltsgenerierung

Neben der RAG-basierten Chat-Funktionalität (siehe Abschnitt [6.8](#)) bietet EduShelf zwei weitere KI-Features: die automatische Generierung von **Lernkarten (Flashcards)** und **Multiple-Choice-Quizzes**. Beide Features nutzen denselben Ollama-LLM-Stack, folgen aber einem eigenständigen Workflow, hierbei wird keine Vektorsuche benötigt, da der Nutzer direkt den Kontext über Dokumenten-IDs oder RAG-abgerufene Chunks liefert.

6.9.1 Generierungspipeline (5 Schritte)

Beide Services (`FlashcardService`, `QuizService`) folgen derselben fünfstufigen Pipeline:

1. **Kontextbeschaffung:** Der Service akzeptiert drei Eingabequellen: einen direkten `Context-String` (aus der RAG-Pipeline), eine einzelne `DocumentId` oder eine Liste von `DocumentIds`. Im letzteren Fall werden die Textinhalte aller Dokumente via `GetDocumentContentAsync` abgerufen und mit einem Trennzeichen zusammengefügt.
2. **Kontext-Truncation:** Um das Kontextfenster des LLMs nicht zu sprengen, wird der Dokumenteninhalt auf maximal 12.000 Zeichen (Flashcards) bzw. 15.000 Zeichen (Quizze) begrenzt.

3. **Prompt-Vorbereitung:** Der System-Prompt wird aus `appsettings.json` geladen. Die Variable `{Count}` wird durch die gewünschte Anzahl ersetzt.
4. **LLM-Inferenz:** Das Modell wird mit dem vorbereiteten `ChatHistory`-Objekt aufgerufen.
5. **JSON-Parsing und Persistenz:** Die Antwort wird geparkt und in der Datenbank gespeichert.

6.9.2 Flashcard-Generierung (FlashcardService)

Der LLM wird angewiesen, eine JSON-Liste von Objekten mit den Feldern `Question` und `Answer` zurückzugeben. Das folgende Snippet zeigt den Prompt-Aufbau und die anschließende Verarbeitung:

```
1 // Prompt zusammenbauen und KI aufrufen
2 var chatHistory = new ChatHistory();
3 chatHistory.AddSystemMessage(
4     promptTemplate.Replace("{Count}", request.Count.ToString()));
5 chatHistory.AddUserMessage(
6     $"Text to analyze:\n\n{documentContent}\n\n" +
7     "IMPORTANT: Output strictly valid JSON. Start with [.]");
8
9 var result = await chatCompletionService
10     .GetChatMessageContentAsync(chatHistory);
11
12 // JSON bereinigen und deserialisieren
13 var cleanedJson = JsonHelper.ExtractJson(result.Content);
14 var flashcards = JsonSerializer.Deserialize<List<GeneratedFlashcardJson>>(
15     cleanedJson,
16     new JsonSerializerOptions { PropertyNameCaseInsensitive = true });
```

Listing 6.9: KI-Aufruf und JSON-Parsing in `FlashcardService.cs`

Nach dem Parsing werden die generierten Karten automatisch mit einem Tag versehen, der den Dokumenttitel trägt, und als `Flashcard`-Entitäten in der Datenbank gespeichert.

6.9.3 Quiz-Generierung (QuizService)

Die Quiz-Generierung ist komplex, da das LLM ein verschachteltes JSON-Objekt mit `Title`, `Questions` und pro Frage mehrere `Answers` (jeweils mit `IsCorrect`-Flag) produzieren muss. Da LLMs trotz strikter Anweisung gelegentlich vom erwarteten Schema abweichen (z.B. alternative Feldnamen wie `options` statt `Answers`), implementiert der `QuizService` eine **zweistufige Parsing-Strategie**:

```

1 var jsonNode = JsonNode.Parse(cleanedJson);
2
3 // Stufe 1: Wrapper-Handling (falls LLM { "quiz": {...} } liefert)
4 if (jsonNode["quiz"] != null)
5     jsonNode = jsonNode["quiz"];
6
7 // Stufe 2a: Standard-Deserialisierung versuchen
8 var result = jsonNode.Deserialize<GeneratedQuizJson>(options);
9
10 // Stufe 2b: Fallback -- Manuelles Parsing bei alternativen Schemas
11 if (result == null || !result.Questions.Any())
12 {
13     // Felder wie "text"/"Text", "options"/"answers", "answer" (string)
14     // werden manuell ausgelesen und normalisiert.
15     result = ManuallyParseQuizJson(jsonNode);
16 }

```

Listing 6.10: Robustes JSON-Parsing in QuizService.cs

Die manuelle Fallback-Stufe erkennt dabei sowohl Antwortoptionen als einfache Strings (`[Paris", London", ...]`) als auch als Objekte (`[{text: ..., isCorrect: ...}]`). Dies macht die Quiz-Generierung robust gegenüber variierenden LLM-Antwortformaten.

Nach erfolgreichem Parsing wird das Quiz als `Quiz`-Entität mit allen `Question`- und `Answer`-Objekten in einer einzigen EF Core-Transaktion gespeichert.

6.10 Infrastruktur & Deployment

Das System ist vollständig containerisiert, um eine konsistente Umgebung von der Entwicklung bis zur Produktion zu gewährleisten [11].

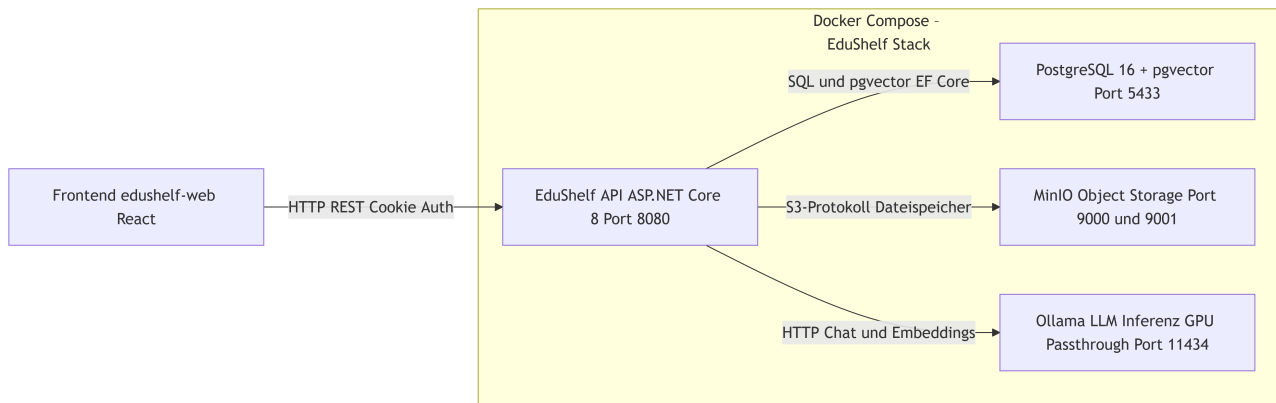


Abbildung 6.6: Infrastrukturdiagramm des EduShelf Docker-Stacks

6.10.1 Docker-Komposition

Die `docker-compose.yml` definiert das Zusammenspiel der Dienste und deren spezifische Konfiguration für die Produktionsumgebung.

6.10.1.1 Datenbank (PostgreSQL mit pgvector)

Für die persistente Speicherung und Vektorsuche wird ein spezialisiertes Image verwendet:

- **Image:** `pgvector/pgvector:pg16` - Dies ist essentiell, da das Standard-Postgres-Image die `vector`-Erweiterung nicht enthält.
- **Ports:** Das Mapping `5433:5432` vermeidet Konflikte mit lokalen Postgres-Instanzen auf dem Host.
- **Healthcheck:** Ein integrierter Healthcheck (`pg_isready`) stellt sicher, dass abhängige Container (wie die API) erst starten, wenn die Datenbank vollständig hochgefahren ist.
- **Persistenz:** Das Volume `postgres_data` sichert die relationalen Daten und Vektor-Indizes über Container-Neustarts hinweg.

6.10.1.2 KI-Inferenz (Ollama)

Der Ollama-Container ist für die lokale Ausführung der LLMs zuständig und benötigt direkten Hardware-Zugriff:

- **GPU-Passthrough:** Über die `deploy.resources`-Sektion wird der Nvidia-Treiber durchgereicht:

```
reservations:
  devices:
    - driver: nvidia
      count: 1
      capabilities: [gpu]
```

Dies ermöglicht die Nutzung von CUDA-Kernen, was die Antwortzeit drastisch reduziert.

- **Persistenz:** Das Volume `ollama_data` speichert die heruntergeladenen Modelle (z.B. Llama 3) im `/root/.ollama` Verzeichnis, um erneute Downloads bei Container-Updates zu vermeiden.
- **Keep-Alive:** Die Umgebungsvariable `OLLAMA_KEEP_ALIVE` verhindert, dass das Modell nach kurzer Inaktivität aus dem VRAM (Video Random Access Memory) entladen wird, was die Latenz bei aufeinanderfolgenden Anfragen minimiert.

6.10.1.3 Object Storage (MinIO)

MinIO dient als skalierbarer, S3-kompatibler Dateispeicher.

- **Ports:**
 - 9000: API-Port für die Kommunikation mit dem Backend (S3-Protokoll).
 - 9001: Web-Interface (Console) zur manuellen Verwaltung von Buckets und Dateien.
- **Command:** Der Startbefehl `server /data -console-address :9001` aktiviert das Web-UI explizit.
- **Volumes:** `minio_data` speichert die physischen Dateien (PDFs, Bilder) persistent.

6.10.2 Dockerfile

Das Dockerfile der API nutzt Multi-Stage Builds, um das Image klein zu halten:

1. **Build Stage:** Nutzt das vollständige SDK Image (`mcr.microsoft.com/dotnet/sdk:8.0`), um den Code zu kompilieren und Abhängigkeiten (NuGet Pakete) zu laden.
2. **Runtime Stage:** Nutzt das Runtime Image (`mcr.microsoft.com/dotnet/aspnet:8.0`), welches nur die zum Ausführen nötigen Komponenten enthält. Das kompilierte Artefakt aus Stage 1 wird hierhin kopiert.

Dies reduziert die Image-Größe drastisch und verbessert die Sicherheit, weil keine Compiler-Tools im Produktions-Container vorhanden sind.

6.11 Resilienz und Stabilität

In verteilten Systemen, insbesondere bei der Anbindung von KI-Diensten, sind temporäre Ausfälle unvermeidbar. Um die Robustheit der Anwendung zu gewährleisten, wurde ein mehrstufiges Resilienz-Konzept implementiert.

6.11.1 Retry-Policies & Circuit Breaker

In der `ServiceCollectionExtensions.cs` werden resiliente HTTP-Clients mithilfe der Bibliothek Polly [\[21\]](#) konfiguriert:

1. **Retry Policy:** Bei Fehlern (z.B. Netzwerk-Glitch, 503 Service Unavailable) wird der Request maximal dreimal mit steigenden Wartezeiten (exponentieller Backoff, 2^n Sekunden) erneut gesendet, bevor ein finaler Fehler ausgegeben wird.
2. **Circuit Breaker:** Wenn 5 Fehler in Folge auftreten, „öffnet“ sich der Stromkreis für 30 Sekunden. Alle weiteren Anfragen werden sofort abgeblockt, um das überlastete Subsystem (z.B. Ollama) zu schonen.

Beide Policies werden auf zwei HTTP-Clients angewandt: den `ImageProcessingService-Client` (für die Vision-basierte Bildanalyse) und den `Ollama-Client` (für LLM-Inferenz und Embeddings). Für den Ollama-Client

wird ein `PolicyHttpClientHandler` manuell konfiguriert, da das Semantic Kernel SDK eine `HttpClient`-Instanz direkt akzeptiert:

```
1 var retryPolicy = HttpPolicyExtensions
2   .HandleTransientHttpError()
3   .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2,
4     retryAttempt)));
5
6 var circuitBreakerPolicy = HttpPolicyExtensions
7   .HandleTransientHttpError()
8   .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));
9
10 // Ollama Client: Retry + Circuit Breaker manuell gewrappt
11 var ollamaHandler = new PolicyHttpClientMessageHandler(
12   retryPolicy.WrapAsync(circuitBreakerPolicy));
13 ollamaHandler.InnerHandler = new HttpClientHandler();
14
15 var ollamaClient = new HttpClient(ollamaHandler)
16 {
17   BaseAddress = new Uri(configuration["AIService:Endpoint"]!),
18   Timeout = TimeSpan.FromMinutes(10)
19 };
```

Listing 6.11: Polly-Konfiguration und HTTP-Client-Setup

6.11.2 Timeout-Konfiguration

Da KI-Operationen (insbesondere Embedding-Generierung für große Dokumente und LLM-Inferenz) erheblich länger dauern können als typische HTTP-Requests, ist der Standard-Timeout von 100 Sekunden nicht ausreichend. Beide HTTP-Clients (`ImageProcessingService` und `Ollama`) sind daher mit einem Timeout von **10 Minuten** konfiguriert (`TimeSpan.FromMinutes(10)`). Dies stellt sicher, dass auch bei der Verarbeitung umfangreicher Dokumente oder komplexer Prompts keine vorzeitigen Abbrüche auftreten.

6.11.3 Graceful Degradation

Mehrere Services implementieren ein kontrolliertes Fehlerverhalten, bei dem das System dem Nutzer eine verständliche Rückmeldung liefert, anstatt vollständig abzustürzen:

- **IntentDetectionService:** Schlägt das JSON-Parsing der LLM-Antwort fehl (z.B. weil das Modell kein valides JSON liefert), wird automatisch auf den Standard-Intent `question` zurückgefallen, anstatt einen Fehler auszulösen.

- **ChatService:** Wird für eine Flashcard- oder Quiz-Generierung kein ausreichender Dokumentenkontext gefunden, antwortet das System mit einer informativen Nachricht anstatt den Request abzuberechnen.
- **BackgroundJobService:** Die Verarbeitungsschleife nutzt eine doppelte `try-catch`-Struktur. Fehler bei der Ausführung einzelner Jobs werden isoliert geloggt, ohne die Verarbeitungsschleife anzuhalten. Zusätzlich wird ein separater `catch`-Block für Dequeue-Fehler verwendet, um sicherzustellen, dass der Service auch bei Queue-Problemen stabil weiterläuft.

6.11.4 Strukturiertes Logging

Alle Services verwenden den integrierten .NET `ILogger` mit differenzierten Log-Levels:

- **LogError:** Für interne Systemfehler wie Datenbank-, KI-Service- oder Dateiverarbeitungsfehler. Diese werden in der `ErrorHandlingMiddleware` und allen Services konsistent verwendet.
- **LogWarning:** Für erwartbare Fehlerzustände wie nicht gefundene Ressourcen (404), ungültige Eingaben (400) oder fehlgeschlagene Authentifizierung (401). Diese Unterscheidung ermöglicht es, in Monitoring-Tools (z.B. Grafana, Seq) gezielt nach kritischen Fehlern zu filtern, ohne von erwartbaren Client-Fehlern überlagert zu werden.

6.12 Herausforderungen und Lösungen

Während der Backend-Entwicklung mussten mehrere technische Hürden überwunden werden.

6.12.1 Blockierende Operationen bei großen Dateien

Problem: Die Verarbeitung großer PDF-Dateien (Textextraktion, Chunking, Embedding-Generierung) innerhalb des HTTP-Request-Zyklus führte zu langen Wartezeiten und Timeouts im Frontend. **Lösung:** Einführung einer `BackgroundJobQueue`. Der Upload-Endpoint nimmt die Datei entgegen, speichert sie in MinIO und reiht nur eine „Job-ID“ in die Queue ein, bevor er sofort `202 Accepted` zurückgibt. Ein Hintergrunddienst („Worker“) arbeitet diese Queue asynchron ab (siehe Abbildung 6.7).

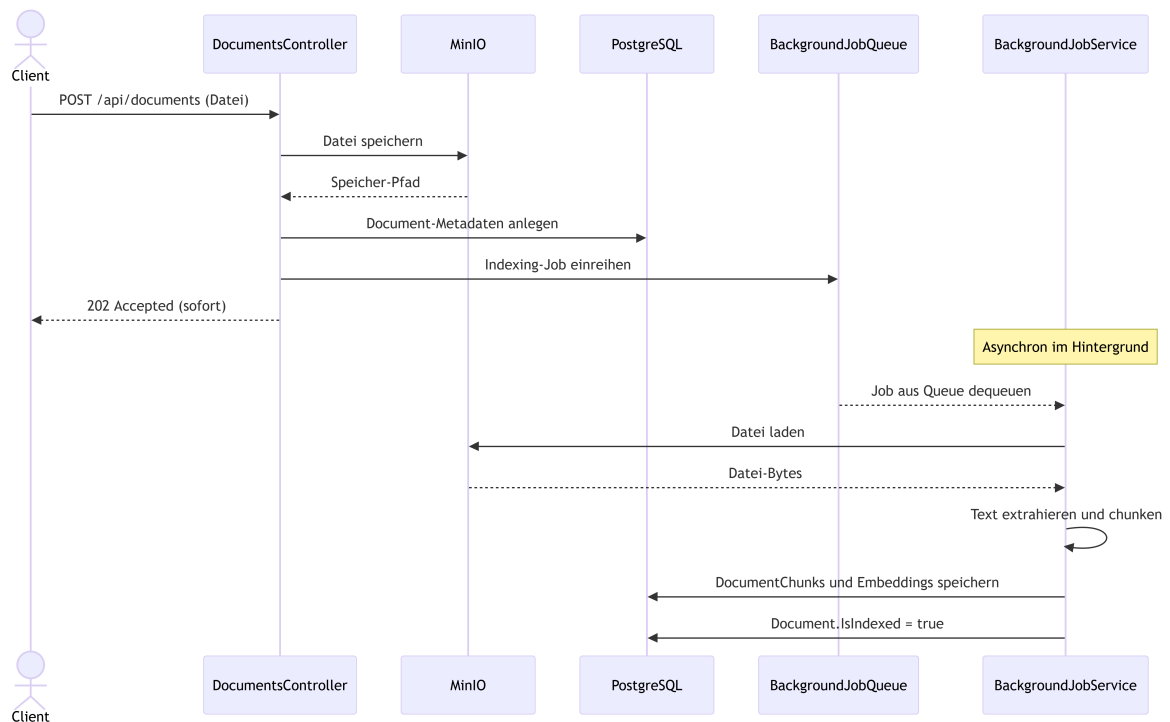


Abbildung 6.7: Sequenzdiagramm des asynchronen Dokument-Upload-Flows

```

1 protected override async Task ExecuteAsync(CancellationToken stoppingToken)
2 {
3     while (!stoppingToken.IsCancellationRequested)
4     {
5         var workItem = await _queue.DequeueAsync(stoppingToken); //
6         // Blockiert bis Arbeit da ist
7         try
8         {
9             await workItem(stoppingToken); // Fuehrt den eigentlichen
10            // Indexing-Task aus
11        }
12        catch (Exception ex)
13        {
14            _logger.LogError(ex, "Error occurred executing background work
15            item.");
16        }
17    }
18 }

```

Listing 6.12: Verarbeitungsschleife im BackgroundJobService

6.12.2 LLM Kontext-Fenster-Limits

Problem: Qwen3 und andere Modelle haben ein festes Limit an Tokens (z.B. 4096 oder 8192). Ein blindes Einfügen von Dokumenten würde zu Abstürzen führen. **Lösung:** Implementierung des `TokenService` mit *Tiktoken*. Bevor ein Prompt generiert wird, werden die Tokens der abgerufenen Chunks gezählt. Ist das Limit erreicht, werden die am wenigsten relevanten Chunks verworfen („Smart Truncation“).

6.12.3 Datenkonsistenz bei Löschoperationen

Problem: Wenn ein User gelöscht wird, dürfen dessen Dokumente und Vektoren nicht als „Datenmüll“ in der Datenbank verbleiben. **Lösung:** Nutzung von `OnDelete(DeleteBehavior.Cascade)` in EF Core. Da auch die Vektoren (`DocumentChunk`) über Foreign Keys hart an das `Document` und dieses an den `User` gekoppelt ist, löscht ein einziger SQL-Befehl (`DELETE FROM Users WHERE Id=...`) den gesamten assoziierten Datenbaum.

6.13 Qualitätssicherung und Tests

Die Stabilität des Backends wird durch automatisierte Tests auf Basis von **xUnit** gewährleistet. Das Testprojekt **EduShelf.Api.Tests** enthält 16 Testklassen, die Geschäftslogik, Controller, Validatoren und Hilfsfunktionen abdecken.

Testklasse	Getestete Komponente
AuthServiceTests	Registrierung, Login, E-Mail-Bestätigung
FlashcardServiceTests	KI-Flashcard-Generierung, JSON-Fehler
QuizServiceTests	KI-Quiz-Generierung, Parsing-Robustheit
ChatServiceTests	RAG-Orchestrierung, Intent-Branching
DocumentServiceTests	Upload, Löschung, Indexierung
IntentDetectionServiceTests	JSON-Intent-Parsing
TokenServiceTests	Token-Limitierung und Truncation
ResilienceTests	Polly Retry- und Circuit-Breaker-Policies
UserRegisterValidatorTests	FluentValidation-Regeln
JsonHelperTests	JSON-Extraktion aus LLM-Antworten
DocumentsControllerTests	HTTP-Routing und Autorisierung
FlashcardsControllerTests	HTTP-Routing und Autorisierung
UsersControllerTests	HTTP-Routing und Autorisierung
TagsControllerTests	HTTP-Routing und RBAC
QuizCreateDtoValidatorTests	Eingabvalidierung für Quiz-DTOs
ChatControllerTests	HTTP-Routing und RAG-Integration

Tabelle 6.3: Übersicht der Test-Klassen im Testprojekt

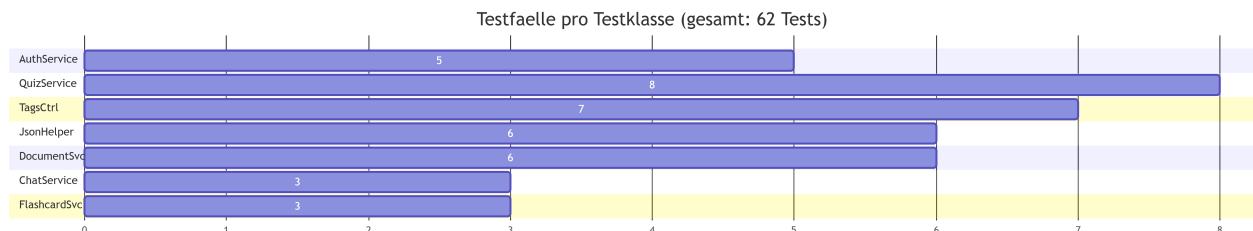


Abbildung 6.8: Anzahl der Unit-Tests pro Testklasse (gesamt: 62 Tests)

6.13.1 Unit Tests (xUnit + Moq)

Die Geschäftslogik (Services) wird isoliert getestet. Externe Abhängigkeiten wie Datenbank, LLM oder E-Mail-Dienst werden mittels **Moq** durch kontrollierte Testdoubles ersetzt. Die Datenbank wird für jeden Testfall durch eine separate In-Memory-Datenbankinstanz (über **UseInMemoryDatabase** mit **Guid.NewGuid()**) vollständig isoliert.

6.13.1.1 Beispiel: FlashcardService

Die drei Kerntests des FlashcardService zeigen das typische Testmuster:

```

1 [Fact]
2 public async Task
3     GenerateFlashcardsAsync_ValidContent_ShouldCreateFlashcards()
4 {
5     // Arrange -- Mock-LLM gibt valides JSON zurueck
6     var jsonResponse = "[{\"Question\": \"What is AI?\", \"Answer\": \"
7         Artificial Intelligence\"},\" +
8         \" {\"Question\": \"Is it cool?\", \"Answer\": \"Yes
9         \"}]\";
10
11     _mockChatCompletionService
12         .Setup(c => c.GetChatMessageContentsAsync(...))
13         .ReturnsAsync(new List<ChatMessageContent> { new(AuthorRole.
14             Assistant, jsonResponse) });
15
16     // Act
17     var result = await service.GenerateFlashcardsAsync(request, userId);
18
19     // Assert
20     Assert.Equal(2, result.ToList().Count);
21     Assert.Equal(2, context.Flashcards.Count());
22     Assert.Single(context.Tags); // Tag "Test Doc" automatisch erstellt
23 }

```

Listing 6.13: Unit Test: Erfolgreiche Flashcard-Generierung

6.13.1.2 Beispiel: AuthService

Der AuthService wird ebenfalls umfassend getestet, inklusive des E-Mail-Bestätigungsflows:

```
1 [Fact]
2 public async Task RegisterUserAsync_ValidData_ShouldCreateUserAndSendEmail()
3 {
4     var result = await service.RegisterUserAsync(registerDto);
5
6     Assert.Single(context.Users);
7     Assert.False((await context.Users.FirstAsync()).IsEmailConfirmed);
8     Assert.NotNull((await context.Users.FirstAsync()).EmailConfirmationToken
9         );
10
11     // Verifiziert: E-Mail wurde genau einmal an die richtige Adresse
12     // gesendet
13     _mockEmailService.Verify(
14         x => x.SendEmailAsync("new@example.com", It.IsAny<string>(), It.
15             IsAny<string>()),
16         Times.Once);
17 }
```

Listing 6.14: Unit Test: Registrierung erstellt Nutzer und sendet E-Mail

6.13.2 Validatoren-Tests

FluentValidation-Regeln werden mit der Bibliotheksmethode `TestValidate` isoliert geprüft:

```

1 [Fact]
2 public void Should_Have_Error_When_Email_Is_Invalid()
3 {
4     var model = new UserRegister { Username = "user", Email = "invalid_email", Password = "pw" };
5     var result = _validator.TestValidate(model);
6     result.ShouldHaveValidationErrorFor(x => x.Email);
7 }
8
9 [Fact]
10 public void Should_Not_Have_Error_When_Model_Is_Valid()
11 {
12     var model = new UserRegister { Username = "validUser", Email = "test@example.com", Password = "strongPassword123" };
13     _validator.TestValidate(model).ShouldNotHaveAnyValidationErrors();
14 }
15

```

Listing 6.15: Validator-Tests mit `FluentValidation.TestHelper`

6.13.3 Integration Tests

Für Controller und die Datenbankinteraktion werden Integrationstests verwendet. Hierbei wird mittels `WebApplicationFactory` ein echter Test-Server im Speicher hochgefahren, der gegen den vollständigen ASP.NET Core Middleware-Stack (Routing, Authentifizierung, Fehlerbehandlung) und eine separate In-Memory-Datenbankinstanz läuft. Dies stellt sicher, dass das Routing, die Autorisierungsattribute und das Entity Framework korrekt zusammenspielen, unabhängig voneinander, aber als zusammenhängendes System.

6.14 Fazit

Das Backend von EduShelf stellt ein robustes, modernes und erweiterungsfähiges Fundament für die Lernplattform dar. Die während der Entwicklung getroffenen Architekturentscheidungen haben sich als funktional erwiesen, um die Anforderungen einer KI-gestützten Dokumentenverwaltungsplattform zu erfüllen.

6.14.1 Erreichte Ziele

- **Vollständige RAG-Pipeline:** Die Kombination aus semantischer Vektorsuche (pgvector + HNSW-Index), Intent-Erkennung und tokenbasierter Kontextlimitierung ermöglicht präzise, dokumentenspezifische KI-Antworten ohne externe API-Abhängigkeiten.
- **Datenschutz durch lokale KI:** Der Einsatz von Ollama als lokaler LLM-Server stellt sicher, dass keine Dokumenteninhalte das eigene System verlassen. Dies ist insbesondere für Bildungseinrichtungen relevant.
- **Robuste Lerninhaltsgenerierung:** Sowohl die Flashcard- als auch die Quiz-Generierung sind durch mehrstufige JSON-Parsing-Strategien widerstandsfähig gegenüber variierenden LLM-Antwortschemas.
- **Sicherheit:** Der vollständige E-Mail-Verifizierungsflow, BCrypt-Passwort-Hashing, HttpOnly-Cookie-Authentifizierung und RBAC schaffen eine mehrschichtige Sicherheitsarchitektur.
- **Infrastruktur:** Die vollständige Containerisierung mit Docker Compose erlaubt ein reproduzierbares, portables Deployment auf jedem Linux-Server ohne manuelles Setup.
- **Teststrategie:** Die Testklassen mit Unit und Integrationstests decken alle sicherheits- und geschäftskritischen Pfade ab.

6.14.2 Lernerkenntnisse und Herausforderungen

Die größte technische Herausforderung war die **Unvorhersehbarkeit von LLM-Ausgaben**: Trotz präziser System-Prompts und expliziter JSON-Formatvorgaben weichen Sprachmodelle gelegentlich vom erwarteten Schema ab. Die Lösung lag nicht im Erzwingen einer bestimmten Ausgabe, sondern in der Implementierung mehrerer Fallback-Strategien im Parsingabschnitt.

Ein weiteres Lernfeld war die **Performance-Optimierung der Ingestion-Pipeline**: Der ursprüngliche Ansatz, Dokumente synchron im HTTP-Request-Zyklus zu verarbeiten, führte bei großen PDFs zu Timeouts. Die Einführung einer **BackgroundJobQueue** mit asynchroner Verarbeitung löste dieses Problem, ohne die Architektur grundlegend zu verändern.

6.14.3 Mögliche Erweiterungen

- **Streaming-Responses:** Die aktuelle Implementierung wartet auf die vollständige LLM-Antwort. Eine WebSocket-Anbindung würde die wahrgenommene Latenz deutlich reduzieren.
- **Multi-Tenant-Unterstützung:** Aktuell ist jeder User vollständig isoliert. Eine optionale Kurs- oder Gruppenebene wäre eine mögliche Erweiterung.
- **Microservice-Migration:** Der aktuelle Ansatz war für die Initialentwicklung ideal. Bei steigender Last könnten rechenintensive Services (Indexierung, KI-Inferenz) als eigenständige Mikroservices ausgelagert werden.

Zeiterfassung

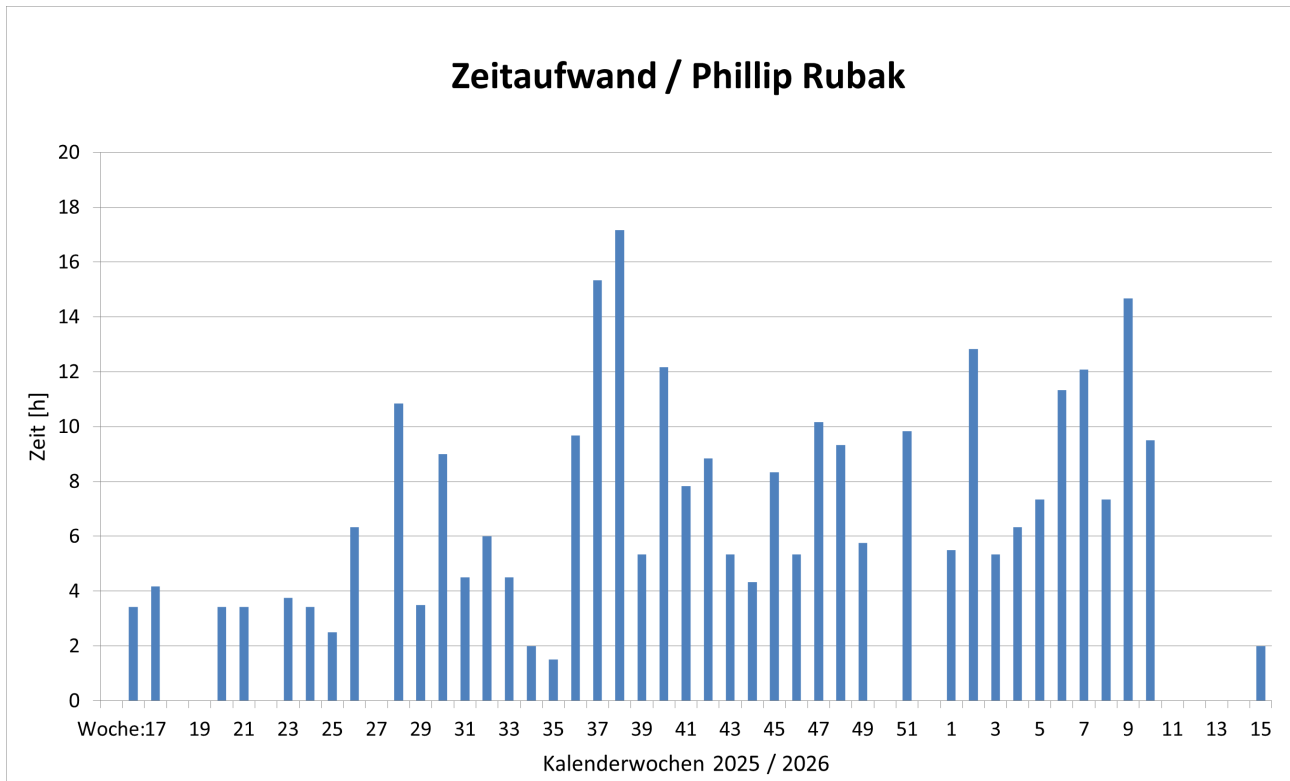


Abbildung 6.9: Backend Zeitaufwand

Gesamt: 301,25h

Freizeit: 234,44h

Quellenverzeichnis

- [1] Meta Platforms, Inc. React – A JavaScript library for building user interfaces.
URL: <https://react.dev/>.
- [2] Evan You and Vite Contributors. Vite – Next Generation Frontend Tooling.
URL: <https://vite.dev/>.
- [3] Mozilla Developer Network. Cross-Origin Resource Sharing (CORS) | MDN.
URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>.
- [4] Remix Software. React Router – Declarative Routing for React.
URL: <https://reactrouter.com/>.
- [5] Mozilla Developer Network. Set-Cookie: HttpOnly – HTTP | MDN.
URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie>.
- [6] Mozilla Developer Network. Window: localStorage property – Web APIs | MDN.
URL: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>.
- [7] Mozilla Developer Network. Promise.all() – JavaScript | MDN.
URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all.
- [8] Mozilla Developer Network. FormData – Web APIs | MDN.
URL: <https://developer.mozilla.org/en-US/docs/Web/API/FormData>.
- [9] Meta Platforms, Inc. Updating Objects in State – React.
URL: <https://react.dev/learn/updating-objects-in-state>.
- [10] Mozilla Developer Network. Using CSS custom properties (variables) | MDN.
URL: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties.
- [11] Docker Documentation.
URL: <https://docs.docker.com/>.
- [12] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [13] Entity Framework Core Documentation.
URL: <https://learn.microsoft.com/en-us/ef/core/>.
- [14] ASP.NET Core Documentation.
URL: <https://learn.microsoft.com/en-us/aspnet/core/>.
- [15] pgvector: Open-source vector similarity search for PostgreSQL.
URL: <https://github.com/pgvector/pgvector>.
- [16] Microsoft Semantic Kernel Documentation.
URL: <https://learn.microsoft.com/en-us/semantic-kernel/overview/>.

- [17] The Twelve-Factor App.
URL: <https://12factor.net/>.
- [18] Yury A. Malkov und Dmitry A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
URL: <https://arxiv.org/abs/1603.09320>.
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel und Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 2020.
URL: <https://arxiv.org/abs/2005.11401>.
- [20] Retrieval Augmented Generation (RAG) für LLMs.
URL: <https://www.promptingguide.ai/de/research/rag>.
- [21] Polly: Resilience and transient-fault-handling library for .NET.
URL: <https://github.com/App-vNext/Polly>.

Abbildungsverzeichnis

4.1	Konzeptionelle Entwicklung der Idee – von der Problemerkennung zur Lösung	31
5.1	Navigationsleiste von EduShelf (Dark-Mode)	38
5.2	Registrierung von EduShelf im Dark-Mode	41
5.3	Login-Ansicht von EduShelf im Dark-Mode	43
5.4	Dashboard von EduShelf im Dark-Mode	44
5.5	Dateiübersicht von EduShelf im Dark-Mode	46
5.6	Share-File-Dialog von EduShelf im Dark-Mode	48
5.7	Übersicht aller verfügbaren Quizzes im Dark-Mode	50
5.8	Quiz-Durchführung mit Antwort-Feedback im Dark-Mode	51
5.9	Quiz-Editor zum Erstellen und Bearbeiten von Quizzes	52
5.10	Lernkarten-Ansicht von EduShelf im Dark-Mode	53
5.11	KI-Chat-Ansicht von EduShelf im Dark-Mode	55
5.12	Chat-Eingabebereich mit Bildupload-Option	56
5.13	Admin-Panel von EduShelf im Dark-Mode	57
5.14	Profilseite von EduShelf im Dark-Mode	59
5.15	Frontend Zeitaufwand	69
6.1	Schichtenarchitektur des EduShelf Backends	71
6.2	Entity Relationship Diagram des EduShelf Backends	77
6.3	Ablaufdiagramm der Ingestion-Pipeline in EduShelf	82
6.4	Architekturdiagramm des RAG-Systems in EduShelf (Quelle: [20])	85
6.5	Intent-basiertes Branching im ChatService	89
6.6	Infrastrukturdiagramm des EduShelf Docker-Stacks	94
6.7	Sequenzdiagramm des asynchronen Dokument-Upload-Flows	100
6.8	Anzahl der Unit-Tests pro Testklasse (gesamt: 62 Tests)	102

6.9 Backend Zeitaufwand	108
-----------------------------------	-----

Code-Verweise

5.1	Auszug aus der Vite-Konfiguration	35
5.2	Routing-Struktur in App.jsx	36
5.3	GET-Methode im API-Service	37
5.4	Outlet-Pattern in MainLayout.jsx	38
5.5	Rollenbasierte Anzeige des Admin-Menüpunkts	39
5.6	Dynamische CSS-Klasse bei aktivem Navigationselement	39
5.7	Clientseitige Validierung der Eingaben	42
5.8	Verarbeitung von Backend-Validierungsfehlern	42
5.9	Paralleles Datenladen mit Promise.all im Dashboard	45
5.10	Clientseitige Echtzeit-Suche in der Dateiliste	46
5.11	Menüsteuerung über activeMenuId	47
5.12	Formularlogik im ShareFileModal	48
5.13	Serverseitige Speicherung der Shares via API	49
5.14	Datenstruktur eines Quiz	50
5.15	Update-Logik für verschachtelte Antworten	52
5.16	CSS für den 3D-Flip-Effekt	53
5.17	Nutzung eines Sets für UI-Zustand	54
5.18	Markdown-Rendering mit Syntax-Highlighting	56
5.19	Loading von Detaildaten erst beim Nutzerklick	58
5.20	Laden gespeicherter Präferenzen	60
5.21	Direkte DOM-Manipulation zur Theme-Umschaltung	61
5.22	Theme-Switching mittels CSS-Variablen	62
5.23	Initialer State im QuizModal	63
5.24	Update-Logik für verschachtelte Antworten	63
5.25	Nutzung eines Sets für UI-State	64
5.26	Asynchrones Laden von Detaildaten	65
5.27	Dockerfile für das Frontend	66
6.1	Claims-Aufbau beim Login in AuthService.cs	79
6.2	Automatische Rollenzuweisung bei der Registrierung	80
6.3	Implementierung der Satz-Segmentierung in TextChunker.cs	83

6.4	Intent-Erkennung via LLM in IntentDetectionService.cs	86
6.5	Dokumenten-spezifische Suche in RetrievalService.cs	87
6.6	Embedding-basierte Vektorsuche in RetrievalService.cs	87
6.7	Prompt-Konstruktion in PromptGenerationService.cs	88
6.8	Intent-basiertes Branching in ChatService.cs	90
6.9	KI-Aufruf und JSON-Parsing in FlashcardService.cs	92
6.10	Robustes JSON-Parsing in QuizService.cs	93
6.11	Polly-Konfiguration und HTTP-Client-Setup	98
6.12	Verarbeitungsschleife im BackgroundJobService	101
6.13	Unit Test: Erfolgreiche Flashcard-Generierung	103
6.14	Unit Test: Registrierung erstellt Nutzer und sendet E-Mail . . .	104
6.15	Validator-Tests mit FluentValidation.TestHelper	105